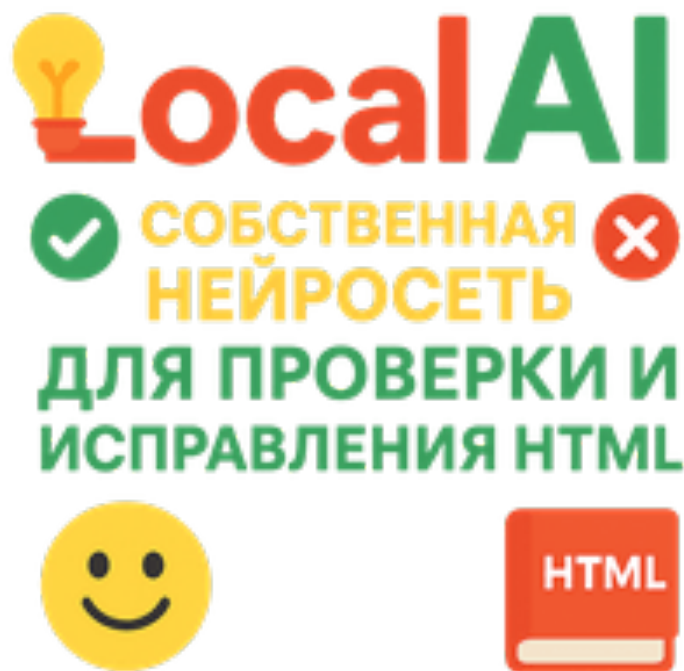




LocalAI — Собственная нейросеть для проверки и исправления HTML

Модуль 1 - Модуль 1. Введение в LocalAI и архитектуру LLM

Урок 1 - Что такое LocalAI и зачем он нужен

Урок 2 - Архитектура LLM и формат моделей GGUF

Урок 3 - Различия между Ollama и LocalAI

Урок 4 - Как LocalAI интегрируется в веб-приложения

Урок 5 - Выбор модели для HTML-валидации

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Алекс создал LocalAI-агент, но при вводе команды «validate html» ничего не происходит. Что первое стоит проверить?

Подсказка: Иногда даже простой синтаксис запуска может быть неверным.

Ответ:

- **Правильность команды (например, «localai validate html»)**
- Сколько памяти выделено процессу

- Версию Python
- Санкции СУБД (база данных)

🔮 Шаг 02:

Миссия 2

🕒 Когда LocalAI отвечает на ваш запрос «исправь HTML», но в выдаче всё ещё есть ошибки, что может быть причиной?

Подсказка: Учёт контекста и правильная конфигурация моделей важны.

Ответ:

- **Неправильная система подсказок (prompt)**
- Нехватка оперативной памяти на машине
- Плохой сетевой кабель
- Ошибка операционной системы

🔮 Шаг 03:

Миссия 3

🕒 При настройке LocalAI ты решил протестировать модель на простом HTML. Какой минимальный файл подойдёт для теста?

Подсказка: Чем проще — тем чище диагностика.

Ответ:

- **<html><body>Hello</body></html>**
- Полный сайт с CSS и JS
- Пустой файл
- Только <!DOCTYPE html>

🔮 Шаг 04:

Миссия 4

🕒 Вы получили сообщение об ошибке «token limit exceeded» при большом HTML. Что лучше сделать?

Подсказка: Разбиение на части — частый метод при больших данных.

Ответ:

- **Разбить HTML на несколько фрагментов и отправлять отдельно**
- Увеличить лимит токенов без разбивки
- Отправить весь HTML как один большой prompt
- Игнорировать ошибку и ждать завершения

🔮 Шаг 05:

Миссия 5

🌐 Твой коллега предлагает использовать внешнюю модель LLM через API вместо локального. В чём преимущество LocalAI?

Подсказка: Подумай про приватность и контроль данных.

Ответ:

- **Полный контроль над данными без передачи сторонним сервисам**
- Более высокая скорость ответа, чем у локальной модели
- Нет необходимости в настройке моделей

- Не требует вычислительных ресурсов

🔮 Шаг 06:

Миссия 6

Сопоставьте понятия LocalAI и LLM с их описаниями:

Ответ:

- LocalAI
- LLM
- Prompt
- Chain-of-Thought

Определения:

1. Большая языковая модель, обученная на тексте
2. Локальный сервер моделей, управляющий LLM на вашей машине
3. Строка запроса к модели, задающая задачу
4. Метод, когда модель «думает вслух», разбивая вывод на шаги

Подсказка: Подумайте, что из названия «LocalAI», а что — метод мышления.

- **LocalAI** — Локальный сервер моделей, управляющий LLM на вашей машине
- **LLM** — Большая языковая модель, обученная на тексте
- **Prompt** — Строка запроса к модели, задающая задачу
- **Chain-of-Thought** — Метод, когда модель «думает вслух», разбивая вывод на шаги

🔮 Шаг 07:

Миссия 7

🔪 Алекс задаёт промпты — сопоставьте стратегию prompt-инжиниринга с её описанием:

Ответ:

- Zero-shot
- One-shot
- Few-shot
- Chain-of-Thought prompt

Определения:

1. Пример отсутствует, просто нужна инструкция
2. Даётся один пример в инструкциях
3. Несколько примеров для ориентира
4. Инструкция включает рассуждения модели шаг за шагом

Подсказка: Смотрите на число примеров, которые вы даёте модели.

- **Zero-shot** — Пример отсутствует, просто нужна инструкция
- **One-shot** — Даётся один пример в инструкциях
- **Few-shot** — Несколько примеров для ориентира
- **Chain-of-Thought prompt** — Инструкция включает рассуждения модели шаг за шагом

🔮 Шаг 08:

Миссия 8

📖 Сопоставьте уровень архитектуры LocalAI с его ролью:

Ответ:

- Frontend-интерфейс
- API-обёртка
- Модель исполнения
- Обработка ошибок и логирование

Определения:

1. Слой между внешними вызовами и ядром
2. Графический интерфейс пользователя или CLI
3. Контроль за исключениями, трассировки, логами
4. Запуск и вычисление модели LLM

Подсказка: Думайте, что именно делает каждый уровень внутри LocalAI.

- **Frontend-интерфейс** — Графический интерфейс пользователя или CLI
- **API-обёртка** — Слой между внешними вызовами и ядром
- **Модель исполнения** — Запуск и вычисление модели LLM
- **Обработка ошибок и логирование** — Контроль за исключениями, трассировки, логами

🔮 Шаг 09:

Миссия 9

✂ Сопоставьте компоненты LocalAI с их задачами:

Ответ:

- Model Hub
- Prompt Manager
- Validator Module
- Error Handler

Определения:

1. Модуль, проверяющий корректность HTML-кода
2. Хранение и управление доступными моделями
3. Управление шаблонами запросов, макросами
4. Обработка исключений и возврат диагностик

Подсказка: Названия часто прямо говорят о функции.

- **Model Hub** — Хранение и управление доступными моделями
- **Prompt Manager** — Управление шаблонами запросов, макросами
- **Validator Module** — Модуль, проверяющий корректность HTML-кода
- **Error Handler** — Обработка исключений и возврат диагностик

🔮 Шаг 10:

Миссия 10

Расположите шаги при реакции LocalAI на запрос «исправь HTML»:

1. Запрос пользователя приходит на API
2. Prompt Manager формирует инструкции
3. Модель LLM генерирует ответ
4. Validator Module анализирует вывод модели
5. Корректировки возвращаются пользователю

Подсказка: Чему предшествует анализ вывода модели?

Ответ:

- Запрос пользователя приходит на API
- Prompt Manager формирует инструкции
- Модель LLM генерирует ответ
- Validator Module анализирует вывод модели
- Корректировки возвращаются пользователю

Шаг 11:

Миссия 11

 Упорядочите стадии оценки корректности HTML LocalAI:

1. Лексический анализ (теги, атрибуты)
2. Проверка вложенности тегов
3. Семантический анализ (идентификаторы, типы атрибутов)
4. Выдача списка ошибок и рекомендаций

Подсказка: Сначала проверяют синтаксис, потом — смысл.

Ответ:

- Лексический анализ (теги, атрибуты)
- Проверка вложенности тегов
- Семантический анализ (идентификаторы, типы атрибутов)
- Выдача списка ошибок и рекомендаций

Шаг 12:

Миссия 12

 Упорядочите шаги отладки промпта, если модель даёт неверные правки:

1. Изменить инструкцию prompt
2. Добавить примеры (few-shot)
3. Проверить формат вывода (JSON, HTML)
4. Проверить, не обрезается ли текст
5. Запустить снова с логированием

Подсказка: Начинают с простого — уточнения инструкции.

Ответ:

- Изменить инструкцию prompt
- Добавить примеры (few-shot)
- Проверить формат вывода (JSON, HTML)
- Проверить, не обрезается ли текст
- Запустить снова с логированием

Шаг 13:

Миссия 13

 Упорядочите этапы внедрения LocalAI в существующий CI/CD:

1. Интеграция вызова LocalAI в pipeline
2. Проверка корректности HTML как этап сборки

3. Обработка результатов с выводом отчётов
4. Откат или остановка сборки при ошибках

Подсказка: CI/CD — это цепочка stages: сначала вызов, затем анализ, потом реакция.

Ответ:

- Интеграция вызова LocalAI в pipeline
- Проверка корректности HTML как этап сборки
- Обработка результатов с выводом отчётов
- Откат или остановка сборки при ошибках

🔮 Шаг 14:

Миссия 14

🗨 После интеграции LocalAI, один из коллег говорит, что «мы теряем гибкость». Как прокомментировать?

Подсказка: Подумайте: автоматизация ≠ жесткость, если есть настройка подсказок.

Ответ:

- **Мы можем настраивать подсказки и режимы, гибкость остаётся**
- **Действительно, гибкости нет** — придётся вручную править
- Мы отключим LocalAI в будущем
- **Коллега прав** — гибкость всегда важнее автоматизации

🔮 Шаг 15:

Миссия 15

🗨 Ты хочешь, чтобы LocalAI сообщал буквально: «в теге нет атрибута alt». Как добиться этого?

Подсказка: Вывод — дело подсказок и шаблонов.

Ответ:

- **Указать шаблон формата выдачи** — детализированный текст
- Увеличить глубину модели
- Добавить CSS-анализ
- Отключить валидацию alt

🔮 Шаг 16:

Миссия 16

🔍 Сопоставьте тип ошибки HTML с примером, который LocalAI мог бы вернуть:

Ответ:

- Отсутствие атрибута alt
- Неправильная вложенность тегов
- Не закрыт тег
- Неподдерживаемый атрибут

Примеры вывода LocalAI:

1. «Тег закрыт после <div>»
2. «У тега отсутствует атрибут alt»
3. «Тег <p> не закрыт»
4. «Атрибут хуз не поддерживается в теге <a>»

Подсказка: Подумайте, какой текст соответствует какому классу ошибки.

- **Отсутствие атрибута alt** — «У тега отсутствует атрибут alt»
- **Неправильная вложенность тегов** — «Тег закрыт после <div>»
- **Не закрыт тег** — «Тег <p> не закрыт»
- **Неподдерживаемый атрибут** — «Атрибут хуз не поддерживается в теге <a>»

🔮 Шаг 17:

Миссия 17

🔥 Последнее испытание! Расположите уровни ответственности при исправлении HTML LocalAI от внешнего к внутреннему:

1. Правила или политики (например, запрет inline-стилей)
2. Шаблон вывода (формат ответа)
3. Промпт-инструкции (как сформулирован запрос)
4. Алгоритм валидации (лексика, вложенность, атрибуты)
5. Выбор модели / конфигурация LLM

Подсказка: Сначала внешние уровни, потом внутренние механики модели.

Ответ:

- Правила или политики (например, запрет inline-стилей)
- Шаблон вывода (формат ответа)
- Промпт-инструкции (как сформулирован запрос)
- Алгоритм валидации (лексика, вложенность, атрибуты)
- Выбор модели / конфигурация LLM

📖 Урок 7 - Викторина 2

🔮 Шаг 01:

Миссия 1

🔮 Ты запускаешь LocalAI и хочешь проверить, работает ли базовая модель корректно. Что первым делом стоит попробовать?

Подсказка: Простая команда без лишних деталей даёт минимальную поверхность для ошибок.

Ответ:

- **Запустить простой prompt «test»**
- Запустить весь сайт через LocalAI
- Отправить пустую строку
- Сразу запустить сложный шаблон с CSS и JS

🔮 Шаг 02:

Миссия 2

Ты обнаружил, что LocalAI иногда игнорирует часть HTML (например, комментарии). Что может быть причиной?

Подсказка: Модели часто обрезают слишком длинные входы.

Ответ:

- **Обрезка prompt'а из-за лимита токенов**
- LocalAI не понимает комментарии
- Модель специально игнорирует комментарии

- Файл комментариев испорчен

🔮 Шаг 03:

Миссия 3

✂ При тестировании ты хочешь, чтобы LocalAI возвращал JSON с полями `errors` и `suggestions`. Что нужно уточнить в промпте?

Подсказка: Формат вывода управляется инструкцией (output template).

Ответ:

- **Добавить в prompt:** «Верни результат в формате JSON с полями errors и suggestions»
- Увеличить мощность модели
- Запросить plain text ответ
- Убрать все ограничители из prompt

🔮 Шаг 04:

Миссия 4

🔧 После обновления модели LocalAI перестал выдавать предложения исправлений, только ошибки. Что логично проверить сначала?

Подсказка: Возможно, отключен вывод suggestions.

Ответ:

- **Настройки шаблона вывода** — sections suggestions отключены
- Модель повреждена
- Лимит токенов слишком мал
- Сеть сломалась

🔮 Шаг 05:

Миссия 5

🕒 Ты хочешь, чтобы LocalAI подчёркивал ошибки прямо в строке (например, указывать позицию). Что стоит добавить в prompt?

Подсказка: Уточните формат «position» или «line:col».

Ответ:

- **Добавить инструкцию:** «Включай позицию в форме line:col»
- Разбить текст построчно
- Увеличить глубину модели
- Убрать формат JSON

🔮 Шаг 06:

Миссия 6

Сопоставьте термин и его роль в архитектуре LocalAI:

Ответ:

- Prompt Template
- Validator
- Postprocessor
- Model Runner

Определения:

1. Модуль, который запускает LLM на основе конфигурации
2. Обрабатывает вывод, форматирует, фильтрует
3. Шаблон для генерации запроса к модели
4. Модуль, который проверяет и маркирует ошибки в выводе

Подсказка: Подумайте, что происходит до и после модельного вывода.

- **Prompt Template** — Шаблон для генерации запроса к модели
- **Validator** — Модуль, который проверяет и маркирует ошибки в выводе
- **Postprocessor** — Обрабатывает вывод, форматирует, фильтрует
- **Model Runner** — Модуль, который запускает LLM на основе конфигурации

Шаг 07:

Миссия 7

 Сопоставьте типы prompt-инжиниринга с их особенностями:

Ответ:

- Instruction only
- Example-based
- Chain-of-Thought
- Self-reflection prompt

Определения:

1. Дает только инструкцию без примеров
2. Включает примеры вместе с инструкцией
3. Просит модель рассуждать шаг за шагом
4. Просит модель оценить и исправить собственный ответ

Подсказка: Сравните, когда используются примеры, а когда — самокритика.

- **Instruction only** — Дает только инструкцию без примеров
- **Example-based** — Включает примеры вместе с инструкцией
- **Chain-of-Thought** — Просит модель рассуждать шаг за шагом
- **Self-reflection prompt** — Просит модель оценить и исправить собственный ответ

Шаг 08:

Миссия 8

 Сопоставьте уровни шаблонов вывода (output templates) с их назначением:

Ответ:

- Минимальный
- Детализированный
- Отладочный
- JSON API

Определения:

1. Возвращает только «error» или «ok»
2. Включает рекомендации, сообщения и позиции
3. Выводит внутренние логи модели, рассуждения и промежуточные шаги
4. Строго структурированный JSON с полями и вложенностями

Подсказка: Подумайте, какой шаблон нужен пользователю, а какой — инженеру.

- **Минимальный** — Возвращает только «error» или «ok»
- **Детализированный** — Включает рекомендации, сообщения и позиции
- **Отладочный** — Выводит внутренние логи модели, рассуждения и промежуточные шаги
- **JSON API** — Строго структурированный JSON с полями и вложенностями

🔮 Шаг 09:

Миссия 9

Упорядочите этапы обработки запроса LocalAI (с точки зрения инженера):

1. Пользователь отправляет HTML + инструкцию
2. Prompt Template строит окончательный текст prompt
3. Model Runner выполняет LLM и получает черновой ответ
4. Validator анализирует черновой ответ, отмечает ошибки
5. Postprocessor форматирует исправления и возвращает ответ

Подсказка: Сначала идёт ввод, потом шаблон, затем модель, валидация и форматирование.

Ответ:

- Пользователь отправляет HTML + инструкцию
- Prompt Template строит окончательный текст prompt
- Model Runner выполняет LLM и получает черновой ответ
- Validator анализирует черновой ответ, отмечает ошибки
- Postprocessor форматирует исправления и возвращает ответ

🔮 Шаг 10:

Миссия 10

🔧 Упорядочите шаги улучшения промпта, если ответы плохие:

1. Добавить примеры (few-shot)
2. Уточнить формулировку задачи (инструкции)
Попросить модель самопроверку (self-reflection)
3. Добавить указание формата ответа
4. Логировать полный вывод модели

Подсказка: Сначала уточняете задачу, затем добавляете структуру и проверку.

Ответ:

- Добавить примеры (few-shot)
- Уточнить формулировку задачи (инструкции)
- Попросить модель самопроверку (self-reflection)
- Добавить указание формата ответа
- Логировать полный вывод модели

🔮 Шаг 11:

Миссия 11

📦 Упорядочите стадии отладочного шаблона (debug template):

1. Вывести внутренние рассуждения модели
2. Показать исходный prompt + шаблон
3. Показать промежуточные шаги генерации

4. Обычный вывод исправленного HTML и ошибок

Подсказка: Debug-шаблон часто расширяет обычный вывод деталями.

Ответ:

- Вывести внутренние рассуждения модели
- Показать исходный prompt + шаблон
- Показать промежуточные шаги генерации
- Обычный вывод исправленного HTML и ошибок

🔮 Шаг 12:

Миссия 12

🔧 Упорядочите слои абстракции, через которые проходит запрос к LocalAI:

1. Frontend (CLI или Web UI)
2. API / HTTP интерфейс
3. Prompt Manager / шаблонизация
4. Model Runner / ядро LLM
5. Validator + Postprocessor

Подсказка: Слой ближе к пользователю идёт первым.

Ответ:

- Frontend (CLI или Web UI)
- API / HTTP интерфейс
- Prompt Manager / шаблонизация
- Model Runner / ядро LLM
- Validator + Postprocessor

🔮 Шаг 13:

Миссия 13

🔧 Упорядочите шаги при автотестировании HTML с LocalAI:

1. Запустить LocalAI внутри CI
2. Отправить тестовый HTML
3. Получить JSON ответа с ошибками и исправлениями
4. Сравнить исправления с эталоном и принять/отклонить
5. Отчёт и лог в интерфейс CI

Подсказка: Порядок: запуск → запрос → ответ → сравнительный анализ → отчёт.

Ответ:

- Запустить LocalAI внутри CI
- Отправить тестовый HTML
- Получить JSON ответа с ошибками и исправлениями
- Сравнить исправления с эталоном и принять/отклонить
- Отчёт и лог в интерфейс CI

🔮 Шаг 14:

Миссия 14

🌐 Упорядочите слои, влияющие на поведение LocalAI в системе с плагинами:

1. Политики плагинов / фильтры
2. Шаблон вывода
3. Prompt / инструкции
4. LLM конфигурация (температура, модель)
5. Validator и postprocessing

Подсказка: Политики — самые внешние, потом шаблоны, инструкции и ядро, потом окончательная проверка.

Ответ:

- Политики плагинов / фильтры
- Шаблон вывода
- Prompt / инструкции
- LLM конфигурация (температура, модель)
- Validator и postprocessing

🔮 Шаг 15:

Миссия 15

🕒 Упорядочите шаги развёртывания LocalAI в продакшене после тестов:

1. Выкатка на staging
2. Запуск smoke-тестов HTML на staging
3. Мониторинг ошибок и логов
4. Переход в production (rollout)
5. Непрерывный аудит HTML и обратная связь

Подсказка: Сначала staging, тесты, мониторинг, затем продакшен и постоянный контроль.

Ответ:

- Выкатка на staging
- Запуск smoke-тестов HTML на staging
- Мониторинг ошибок и логов
- Переход в production (rollout)
- Непрерывный аудит HTML и обратная связь

🔮 Шаг 16:

Миссия 16

🔍 Финальное испытание: упорядочите уровень влияния на результат исправления HTML от наиболее внешнего до внутреннего:

1. Политики безопасности или бизнес-правила
2. Шаблон вывода / формат структуры ответа
3. Инструкция / prompt
4. LLM конфигурация / модель
5. Validator / алгоритм проверки HTML

Подсказка: Политики выше, алгоритм проверки — глубоко внутри.

Ответ:

- Политики безопасности или бизнес-правила
- Шаблон вывода / формат структуры ответа

- Инструкция / prompt
- LLM конфигурация / модель
- Validator / алгоритм проверки HTML

Модуль 2 - Модуль 2. Установка и настройка окружения

Урок 1 - Установка Docker и зависимостей

Урок 2 - Настройка контейнера LocalAI

Урок 3 - Загрузка моделей Mistral, CodeLlama, Llama3

Урок 4 - Проверка API и базовые запросы

Урок 5 - Практика первый запрос для исправления HTML

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

 Ты скачал архив LocalAI и запускаешь первый контейнер. Какой файл обычно отвечает за конфигурацию окружения?

Подсказка: Часто применяется файл с расширением .env.

Ответ:

- **.env**
- config.yaml
- settings.py
- docker-compose.override

Шаг 02:

Миссия 2

При вызове `docker-compose up` ты видишь, что контейнер LocalAI падает с ошибкой «port already in use». Что логично сделать?

Подсказка: Конфликт портов — распространённая причина падения контейнера.

Ответ:

- **Изменить порт в конфигурации или освободить занятый порт**
- Перезапустить Docker демон
- Удалить тома Docker
- Обновить образ LocalAI

Шаг 03:

Миссия 3

 Ты хочешь, чтобы контейнер LocalAI автоматически перезапускался при сбоях. Какой ключ Docker-Compose поможет?

Подсказка: Ищи опцию, связанная с «restart».

Ответ:

- **restart: always**

- auto_restart: true
- on_failure: yes
- reboot: true

🔧 Шаг 04:

Миссия 4

📦 Ты хочешь установить секретный ключ API в контейнер LocalAI, но не жёстко прописывать его в коде. Что лучше сделать?

Подсказка: Использование переменных окружения — безопаснее, чем хардкод.

Ответ:

- **Передавать через переменную окружения (ENV)**
- Хардкодить в Dockerfile
- Сохранять ключ в комментариях
- Паковать ключ в image слоём

🔧 Шаг 05:

Миссия 5

📦 После сборки контейнера ты хочешь проверить, что LocalAI запущен правильно. Что ты можешь сделать внутри контейнера?

Подсказка: Часто проверяют статус процесса или прослушиваемый порт.

Ответ:

- **Запустить команду типа netstat / ps, посмотреть прослушиваемый порт**
- Удалить контейнер
- Сразу отправить запрос API
- Открыть браузер из контейнера

🔧 Шаг 06:

Миссия 6

Сопоставьте компоненты окружения LocalAI с их ролями:

Ответ:

- Dockerfile
- docker-compose.yml
- .env файл
- volume / том Docker

Определения:

1. Определяет, как собрать образ LocalAI
2. Определяет, какие сервисы и связи запустить
3. Содержит секреты и переменные окружения
4. Постоянное хранилище данных между перезапусками

Подсказка: Что делает каждый файл/ресурс в Docker-экосистеме?

- **Dockerfile** — Определяет, как собрать образ LocalAI
- **docker-compose.yml** — Определяет, какие сервисы и связи запустить
- **.env файл** — Содержит секреты и переменные окружения
- **volume / том Docker** — Постоянное хранилище данных между перезапусками

🔧 Шаг 07:

Миссия 7

🔗 Сопоставьте параметры Docker-Compose с их функциями:

Ответ:

- ports
- environment
- volumes
- depends_on

Определения:

1. Какие порты внутри контейнера мапятся наружу
2. Переменные окружения, передаваемые контейнеру
3. Привязка директорий хоста к контейнеру или томов
4. Указывает, что один сервис должен запуститься после другого

Подсказка: Подумайте, как устанавливаются связи и зависимости между сервисами.

- **ports** — Какие порты внутри контейнера мапятся наружу
- **environment** — Переменные окружения, передаваемые контейнеру
- **volumes** — Привязка директорий хоста к контейнеру или томов
- **depends_on** — Указывает, что один сервис должен запуститься после другого

🔧 Шаг 08:

Миссия 8

🌐 Сопоставьте конфигурационные файлы и их назначение в LocalAI окружении:

Ответ:

- config.yaml
- logging.conf
- models.json
- credentials.json

Определения:

1. Общая конфигурация работы LocalAI (порты, режимы)
2. Настройки логирования (формат, уровни)
3. Информация о доступных моделях и их путях
4. Секреты доступа к API и ключи

Подсказка: Названия файлов часто отражают их назначение.

- **config.yaml** — Общая конфигурация работы LocalAI (порты, режимы)
- **logging.conf** — Настройки логирования (формат, уровни)
- **models.json** — Информация о доступных моделях и их путях
- **credentials.json** — Секреты доступа к API и ключи

🔧 Шаг 09:

Миссия 9

🔍 Ты сталкиваешься с проблемой: контейнер не находит модель. Сопоставьте возможные причины с их симптомами:

Ответ:

- Неверный путь к модели
- Несоответствие версии модели и LocalAI
- Файл модели повреждён
- Права доступа (filesystem)

Симптомы:

1. Ошибка «model not found»
2. Ошибка формата или сериализации модели
3. Ошибка чтения файла /permission denied
4. Неожидаемое поведение при генерации

Подсказка: Сообщения об ошибках часто указывают на проблему.

- **Неверный путь к модели** — Ошибка «model not found»
- **Несоответствие версии модели и LocalAI** — Ошибка формата или сериализации модели
- **Файл модели повреждён** — Неожидаемое поведение при генерации
- **Права доступа (filesystem)** — Ошибка чтения файла /permission denied

 Шаг 10:**Миссия 10**

 Расположите шаги установки LocalAI через Docker по порядку:

1. Создать конфигурационные файлы (.env, config.yaml)
2. Написать Dockerfile или взять готовый образ
3. Составить docker-compose.yml с сервисами
4. Запустить `docker-compose up`
5. Проверить лог контейнера и API endpoint

Подсказка: Сначала готовят конфигурации, затем пишут описание, запускают и тестируют.

Ответ:

- Создать конфигурационные файлы (.env, config.yaml)
- Написать Dockerfile или взять готовый образ
- Составить docker-compose.yml с сервисами
- Запустить `docker-compose up`
- Проверить лог контейнера и API endpoint

 Шаг 11:**Миссия 11**

 Упорядочите фазы обновления контейнера LocalAI:

1. Остановить старый контейнер
2. Скачать новый образ или пересобрать
3. Запустить новый контейнер с теми же настройками
4. Проверить работоспособность (логи, API)

Подсказка: Метод «stop → rebuild → run → test» — стандарт для контейнеров.

Ответ:

- Остановить старый контейнер
- Скачать новый образ или пересобрать

- Запустить новый контейнер с теми же настройками
- Проверить работоспособность (логи, API)

🔮 Шаг 12:

Миссия 12

🔧 Упорядочите действия при устранении ошибок запуска контейнера:

1. Посмотреть логи контейнера (docker logs)
2. Проверить конфигурационные файлы на синтаксис
3. Проверить порты и конфликты
4. Проверить права файлов и путь к модели
5. Перезапустить контейнер с отладкой

Подсказка: Сначала читают логи, затем проверяют конфиги, порты, права, затем делают перезапуск.

Ответ:

- Посмотреть логи контейнера (docker logs)
- Проверить конфигурационные файлы на синтаксис
- Проверить порты и конфликты
- Проверить права файлов и путь к модели
- Перезапустить контейнер с отладкой

🔮 Шаг 13:

Миссия 13

🌐 Упорядочите уровни, затрагиваемые настраиванием окружения:

1. Секреты / .env переменные
2. Конфигурация сервиса (config.yaml)
3. Docker образ / сборка
4. Сетевые и портовые настройки
5. Хранилище данных / тома

Подсказка: Секреты и конфиги — самые внешние, затем сборка, сеть, тома.

Ответ:

- Секреты / .env переменные
- Конфигурация сервиса (config.yaml)
- Docker образ / сборка
- Сетевые и портовые настройки
- Хранилище данных / тома

🔮 Шаг 14:

Миссия 14

🔧 Упорядочите шаги, если ты меняешь версию модели в окружении:

1. Остановить контейнер
2. Обновить конфигурацию модели (models.json или путь)
3. Пересобрать или подтянуть образ с новой моделью
4. Запустить контейнер вновь
5. Проверить, что модель загружается корректно

Подсказка: Сначала — остановка, затем обновление, потом запуск и проверка.

Ответ:

- Остановить контейнер
- Обновить конфигурацию модели (models.json или путь)
- Пересобрать или подтянуть образ с новой моделью
- Запустить контейнер вновь
- Проверить, что модель загружается корректно

Шаг 15:

Миссия 15

🔧 Упорядочите этапы безопасного резервного копирования данных томов перед обновлением:

1. Остановить контейнеры, использующие том
2. Создать копию тома (docker cp / tar)
3. Проверить целостность бэкапа
4. Запустить контейнеры вновь

Подсказка: Обязательно останавливать контейнеры до копирования данных.

Ответ:

- Остановить контейнеры, использующие том
- Создать копию тома (docker cp / tar)
- Проверить целостность бэкапа
- Запустить контейнеры вновь

Шаг 16:

Миссия 16

📋 Упорядочите действия мониторинга после запуска окружения:

1. Собирать логи контейнера через bind-mount или том
2. Проверять статус API-запросов (health endpoint)
3. Следить за использованием ресурсов (CPU, память)
4. Настраивать алерты и уведомления

Подсказка: Логирование и health — базовый мониторинг, затем ресурсы и алерты.

Ответ:

- Собирать логи контейнера через bind-mount или том
- Проверять статус API-запросов (health endpoint)
- Следить за использованием ресурсов (CPU, память)
- Настраивать алерты и уведомления

Шаг 17:

Миссия 17

✓ Финальное упражнение: расположите уровни, влияющие на работу окружения LocalAI, от внешнего к внутреннему:

1. Переменные окружения / секреты
2. Конфигурационные файлы (yaml, json)
3. Docker сборка / образы
4. Связи контейнеров, сети, порты

5. Тома и файловая система внутри контейнеров

Подсказка: Внешнее — то, что вы контролируете из вне; внутреннее — данные внутри контейнеров.

Ответ:

- Переменные окружения / секреты
- Конфигурационные файлы (yaml, json)
- Docker сборка / образы
- Связи контейнеров, сети, порты
- Тома и файловая система внутри контейнеров

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Ты устанавливаешь LocalAI локально и видишь инструкцию «pip install localai». Что ты сделаешь первым?

Подсказка: Убедись, что используешь правильное виртуальное окружение.

Ответ:

- **Активировать виртуальное окружение (venv, conda)**
- Запустить `pip install` глобально
- Скачать исходники вручную
- Игнорировать инструкции и сразу запустить LocalAI

Шаг 02:

Миссия 2

🔍 При запуске `localai --version` вы получаете ошибку «command not found». Что наиболее вероятно?

Подсказка: Проверь, где установлен исполняемый файл.

Ответ:

- **Путь к исполняемому файлу не в PATH**
- LocalAI не установлен
- Ошибка в самом LocalAI
- Неправильная версия Python

Шаг 03:

Миссия 3

🔗 Ты клонировал репозиторий LocalAI и хочешь сразу запустить пример. Что важно сделать перед `python main.py`?

Подсказка: Часто нужен `requirements.txt` или дополнительная установка зависимостей.

Ответ:

- **Установить зависимости через `pip install -r requirements.txt`**
- Запустить main.py сразу
- Удалить файл requirements.txt
- Запустить Docker, игнорируя локальную версию

🔧 Шаг 04:

Миссия 4

📖 Ты хочешь, чтобы результаты LocalAI сохранялись между запусками. Какой способ стоит применить?

Подсказка: Использование persistent storage или томов помогает сохранять состояние.

Ответ:

- **Смонтировать volume / persistent storage**
- Каждый раз пересоздавать контейнер без данных
- Записывать в tmpfs
- Сохранять в память процесса

🔧 Шаг 05:

Миссия 5

После установки LocalAI ты запускаешь тестовый HTML и видишь пустой ответ. Что полезно сделать?

Подсказка: Проверь логи приложения — они часто подсказывают, что не так.

Ответ:

- **Открыть логи LocalAI / stderr**
- Игнорировать и отправить другой HTML
- Обновить систему
- Удалить LocalAI и установить заново

🔧 Шаг 06:

Миссия 6

🔗 Сопоставьте команды и их назначение при настройке LocalAI:

Ответ:

- pip install
- docker build
- docker run
- localai serve

Назначения:

1. Запуск встроенного сервера LocalAI
2. Установка пакета в окружение Python
3. Построение Docker-образа
4. Запуск контейнера из образа

Подсказка: Подумайте, что делает каждая команда в процессе установки.

- **pip install** — Установка пакета в окружение Python
- **docker build** — Построение Docker-образа
- **docker run** — Запуск контейнера из образа
- **localai serve** — Запуск встроенного сервера LocalAI

🔧 Шаг 07:

Миссия 7

🔗 Сопоставьте типы файлов конфигурации с их назначением в установке LocalAI:

Ответ:

- requirements.txt
- Dockerfile
- docker-compose.yml
- .env

Назначения:

1. Зависимости Python-пакетов
2. Определяет контейнер и инструкции сборки
3. Описывает, как запускать несколько сервисов вместе
4. Содержит секреты и переменные окружения

Подсказка: Названия файлов часто однозначно указывают на их тип.

- **requirements.txt** — Зависимости Python-пакетов
- **Dockerfile** — Определяет контейнер и инструкции сборки
- **docker-compose.yml** — Описывает, как запускать несколько сервисов вместе
- **.env** — Содержит секреты и переменные окружения

🔗 Шаг 08:

Миссия 8

✂ Сопоставьте причины ошибки «permission denied» при доступе к файлу модели и соответствующие решения:

Ответ:

- Файл принадлежит root
- Нет права на чтение
- Селинукс / AppArmor блокирует доступ
- Путь к файлу неверен

Решения:

1. Изменить владельца или группу
2. Добавить флаг чтения (chmod)
3. Отключить или настроить политику безопасности
4. Указать правильный путь или исправить конфигурацию

Подсказка: Ошибки доступа часто связаны с правами или безопасностью ОС.

- **Файл принадлежит root** — Изменить владельца или группу
- **Нет права на чтение** — Добавить флаг чтения (chmod)
- **Селинукс / AppArmor блокирует доступ** — Отключить или настроить политику безопасности
- **Путь к файлу неверен** — Указать правильный путь или исправить конфигурацию

🔗 Шаг 09:

Миссия 9

🔗 Сопоставьте типы ошибок запуска LocalAI с вероятной причиной:

Ответ:

- ModuleNotFoundError

- ImportError (library mismatch)
- RuntimeError model loading
- BindingError port in use

Причины:

1. Неустановленная зависимость
2. Несовместимая версия библиотеки
3. Некорректный файл модели или отсутствует модель
4. Порт уже используется другим процессом

Подсказка: Текст исключения часто прямо подсказывает причину.

- **ModuleNotFoundError** — Неустановленная зависимость
- **ImportError (library mismatch)** — Несовместимая версия библиотеки
- **RuntimeError model loading** — Некорректный файл модели или отсутствует модель
- **BindingError port in use** — Порт уже используется другим процессом

Шаг 10:

Миссия 10

 Расположите шаги развёртывания LocalAI из исходников в правильном порядке:

1. Клонировать репозиторий
2. Установить зависимости
3. Настроить конфигурационные файлы
4. Запустить LocalAI сервер
5. Проверить работу через API-запрос

Подсказка: Сначала код + зависимости, потом конфигурация и пробный запрос.

Ответ:

- Клонировать репозиторий
- Установить зависимости
- Настроить конфигурационные файлы
- Запустить LocalAI сервер
- Проверить работу через API-запрос

Шаг 11:

Миссия 11

 Упорядочите шаги запуска LocalAI в Docker-окружении:

1. Написать Dockerfile
2. Создать docker-compose.yml
3. Собрать образ (docker build)
4. Запустить контейнер (docker-compose up)
5. Проверить работоспособность

Подсказка: Стандартный поток: конфигурация → сборка → запуск → проверка.

Ответ:

- Написать Dockerfile
- Создать docker-compose.yml
- Собрать образ (docker build)

- Запустить контейнер (docker-compose up)
- Проверить работоспособность

🔧 Шаг 12:

Миссия 12

🔧 Упорядочите шаги диагностики, если LocalAI не запускается:

1. Посмотреть логи запуска
2. Проверить конфигурационные файлы на ошибки
3. Проверить зависимости / версии библиотек
4. Проверить права доступа к файлам
5. Перезапустить приложение

Подсказка: Логи дают подсказку, затем анализ конфигурации, окружения и наконец перезапуск.

Ответ:

- Посмотреть логи запуска
- Проверить конфигурационные файлы на ошибки
- Проверить зависимости / версии библиотек
- Проверить права доступа к файлам
- Перезапустить приложение

🔧 Шаг 13:

Миссия 13

🔧 Упорядочите этапы использования томов (volumes) для сохранения данных:

1. Объявить volume в docker-compose
2. Связать volume с каталогом внутри контейнера
3. Остановить контейнер
4. Перезапустить контейнер с volume
5. Проверить доступность сохранённых данных

Подсказка: Сначала конфигурация, затем остановка, запуск и проверка.

Ответ:

- Объявить volume в docker-compose
- Связать volume с каталогом внутри контейнера
- Остановить контейнер
- Перезапустить контейнер с volume
- Проверить доступность сохранённых данных

🔧 Шаг 14:

Миссия 14

🔧 Упорядочите шаги мониторинга и логирования при запуске окружения:

1. Настроить логирование (logging.conf или аналог)
2. Прописать mount для логов или stdout→host
3. Запустить LocalAI
4. Собирать логи и проверять health endpoint

Подсказка: Процесс: настроить лог, подключить, запустить, проверять.

Ответ:

- Настроить логирование (logging.conf или аналог)
- Прописать mount для логов или stdout→host
- Запустить LocalAI
- Собирать логи и проверять health endpoint

 Шаг 15:

Миссия 15

Упорядочите элементы, которые нужно проверить перед запуском LocalAI в production:

1. Проверить переменные окружения / секреты
2. Проверить конфигурацию (yaml, json)
3. Убедиться, что зависимости установлены и версии совместимы
4. Настроить тома / volume для данных и логов
5. Убедиться, что метрики / health endpoint работают

Подсказка: Сначала — окружение, потом — конфигурация, зависимости, хранилище и мониторинг.

Ответ:

- Проверить переменные окружения / секреты
- Проверить конфигурацию (yaml, json)
- Убедиться, что зависимости установлены и версии совместимы
- Настроить тома / volume для данных и логов
- Убедиться, что метрики / health endpoint работают

 Шаг 16:

Миссия 16

 Упорядочите шаги обновления окружения LocalAI без потери данных:

1. Остановить контейнеры
2. Резервное копирование томов / данных
3. Обновить образ / код
4. Запустить новые контейнеры с теми же volume
5. Проверить работоспособность и целостность данных

Подсказка: Обязательно сделать бэкап перед обновлением.

Ответ:

- Остановить контейнеры
- Резервное копирование томов / данных
- Обновить образ / код
- Запустить новые контейнеры с теми же volume
- Проверить работоспособность и целостность данных

 Шаг 17:

Миссия 17

✓ Финальное задание: расположите уровни влияния при настройке окружения LocalAI от внешнего к внутреннему:

1. Секреты / переменные окружения

2. Конфигурационные файлы (yaml / json)
3. Сборка / образ Docker или установка Python
4. Связи контейнеров / сеть / порты
5. Тома и файловая система внутри контейнера

Подсказка: Внешний уровень — то, что вы меняете вручную, внутренний — то, что остаётся внутри контейнера.

Ответ:

- Секреты / переменные окружения
- Конфигурационные файлы (yaml / json)
- Сборка / образ Docker или установка Python
- Связи контейнеров / сеть / порты
- Тома и файловая система внутри контейнера

Модуль 3 - Модуль 3. Настройка модели для HTML-валидации

 **Урок 1 - Создание конфигурации модели htmlfixer.yaml**

 **Урок 2 - Использование системных инструкций (prompt engineering)**

 **Урок 3 - Параметры temperature, num_ctx, stopwords**

 **Урок 4 - Настройка модели под эксперта HTML5**

 **Урок 5 - Формирование шаблона ответов (код + комментарии)**

 **Урок 6 - Викторина 1**

 **Шаг 01:**

Миссия 1

Ты хочешь, чтобы модель LocalAI проверяла HTML на ошибки базового синтаксиса. Какой режим проверки стоит указать?

Подсказка: Начинаем с самого базового уровня — синтаксического.

Ответ:

- **Syntax validation**
- Semantic validation
- Accessibility check
- Performance analysis

 **Шаг 02:**

Миссия 2

Ты хочешь, чтобы модель действовала осторожно и не переписала весь документ. Какой параметр стоит понизить?

Подсказка: Контролировать степень изменений — хорошая стратегия.

Ответ:

- **Change aggressiveness / rewriting threshold**
- Temperature

- Beam width
- Max tokens

🔮 Шаг 03:

Миссия 3

🌐 Ты хочешь, чтобы модель знала о специфике HTML5. Что добавить в prompt?

Подсказка: Уточнение стандарта помогает модели понимать особенности тегов.

Ответ:

- **Уточнить: "Use HTML5 standards"**
- Уточнить: "Use XHTML rules"
- Уточнить: "Use XML syntax"
- Уточнить: "Use HTML 4 rules"

🔮 Шаг 04:

Миссия 4

♥ Ты хочешь, чтобы модель игнорировала inline-стили и не предлагала их удалять. Что стоит указать?

Подсказка: Подскажи правила, которые модель не должна нарушать.

Ответ:

- **Добавить правило: «Не трогать inline styles»**
- Запретить CSS вообще
- Запретить весь атрибут style
- Удалить все стили в output

🔮 Шаг 05:

Миссия 5

♾ Ты хочешь, чтобы модель выдавала результат с минимальным риском ошибки. Какой параметр полезно уменьшить?

Подсказка: Контроль неопределённости часто через «temperature».

Ответ:

- **Установить низкую температуру (например, 0.0-0.2)**
- Увеличить max tokens
- Уменьшить beam width
- Увеличить жесткость правил

🔮 Шаг 06:

Миссия 6

Сопоставьте параметр модели и его роль в HTML-валидации:

Ответ:

- temperature
- top_p
- max_tokens
- rewriting threshold

Роли:

1. Контролирует, насколько сильно модель может изменять документ
2. Ограничивает длину генерируемого текста
3. Управляет случайностью ответов
4. Задаёт лимит вероятностного выбора слова

Подсказка: temperature и top_p часто идут вместе.

- **temperature** — Управляет случайностью ответов
- **top_p** — Задаёт лимит вероятностного выбора слова
- **max_tokens** — Ограничивает длину генерируемого текста
- **rewriting threshold** — Контролирует, насколько сильно модель может изменять документ

Шаг 07:

Миссия 7

 Сопоставьте тип шаблона (template) с его назначением:

Ответ:

- Minimal template
- Verbose template
- Debug template
- Strict template

Назначения:

1. Выдаёт лишь “errors” без пояснений
2. Включает объяснения, предложения и позиции ошибок
3. Включает внутренние логи модели и рассуждения
4. Не допускает изменений, только указывает ошибки

Подсказка: Шаблоны несут от простого к детальному.

- **Minimal template** — Выдаёт лишь “errors” без пояснений
- **Verbose template** — Включает объяснения, предложения и позиции ошибок
- **Debug template** — Включает внутренние логи модели и рассуждения
- **Strict template** — Не допускает изменений, только указывает ошибки

Шаг 08:

Миссия 8

 Сопоставьте режим валидации и его особенности:

Ответ:

- Syntax only
- Semantic + attributes
- Accessibility
- Custom rule set

Особенности:

1. Проверяет структуру тегов и синтаксис
2. Проверяет вложенность, атрибуты, типы значений
3. Проверяет ARIA, alt, правила доступности
4. Включает пользовательские правила и исключения

Подсказка: Подумайте, что сверх базового синтаксиса может проверяться.

- **Syntax only** — Проверяет структуру тегов и синтаксис
- **Semantic + attributes** — Проверяет вложенность, атрибуты, типы значений
- **Accessibility** — Проверяет ARIA, alt, правила доступности
- **Custom rule set** — Включает пользовательские правила и исключения

🔗 Шаг 09:

Миссия 9

🔍 Сопоставьте ошибку, которую модель может выдать, с ее корректным описанием:

Ответ:

- Unclosed tag
- Invalid attribute value
- Missing alt on img
- Tag nesting violation

Описание:

1. Тег не закрыт соответствующим закрывающим
2. Атрибут имеет недопустимое значение или формат
3. У тега `img` отсутствует атрибут `alt`
4. Тег вложен в запрещённый или неверный родитель

Подсказка: Ошибки часто прямо описывают проблему.

- **Unclosed tag** — Тег не закрыт соответствующим закрывающим
- **Invalid attribute value** — Атрибут имеет недопустимое значение или формат
- **Missing alt on img** — У тега `img` отсутствует атрибут `alt`
- **Tag nesting violation** — Тег вложен в запрещённый или неверный родитель

🔗 Шаг 10:

Миссия 10

🔧 Расположите шаги настройки модели для HTML-валидации:

1. Выбрать тип валидации (syntax, semantic и т.д.)
2. Установить параметры `temperature` / `top_p` / `max_tokens`
3. Выбрать шаблон вывода (template)
4. Уточнить правила (например, запрет inline-стилей)
5. Запустить тестовый HTML и проверить результат

Подсказка: Сначала определяют цели, затем настраивают параметры, далее правила, и только в конце — тест.

Ответ:

- Выбрать тип валидации (syntax, semantic и т.д.)
- Установить параметры `temperature` / `top_p` / `max_tokens`
- Выбрать шаблон вывода (template)
- Уточнить правила (например, запрет inline-стилей)
- Запустить тестовый HTML и проверить результат

Шаг 11:

Миссия 11

🔧 Упорядочите этапы отладки, если модель выдает неверные исправления:

1. Просмотреть логи модели / output
2. Понизить temperature / ужесточить правила
3. Изменить шаблон вывода
4. Добавить примеры в prompt (few-shot)
5. Запустить снова и сравнить результаты

Подсказка: Сначала диагностируйте, потом настраивайте, и только потом тестируйте снова.

Ответ:

- Просмотреть логи модели / output
- Понизить temperature / ужесточить правила
- Изменить шаблон вывода
- Добавить примеры в prompt (few-shot)
- Запустить снова и сравнить результаты

Шаг 12:

Миссия 12

📄 Расположите слои, которые влияют на поведение модели при валидации HTML:

1. Тип валидации (syntax vs semantic vs custom)
2. Параметры модели (temperature, max_tokens и др.)
3. Шаблон вывода (template)
4. Дополнительные правила / фильтры
5. Пост-обработка и финальные коррекции (postprocessor)

Подсказка: Чем ближе уровень к ядру, тем глубже он влияет на результат.

Ответ:

- Тип валидации (syntax vs semantic vs custom)
- Параметры модели (temperature, max_tokens и др.)
- Шаблон вывода (template)
- Дополнительные правила / фильтры
- Пост-обработка и финальные коррекции (postprocessor)

Шаг 13:

Миссия 13

🔍 Упорядочите шаги применения пользовательских правил (custom rules) в процессе валидации:

1. Определить формат правил (yaml / JSON)
2. Встроить правила в prompt или шаблон
3. Запуск модели с этими правилами
4. Сопоставление выводов модели с пользовательскими правилами
5. Применить корректировки на основе этих правил

Подсказка: Сначала — формализация, затем интеграция, потом проверка и применение.

Ответ:

- Определить формат правил (yaml / JSON)

- Встроить правила в prompt или шаблон
- Запуск модели с этими правилами
- Сопоставление выводов модели с пользовательскими правилами
- Применить корректировки на основе этих правил

Шаг 14:

Миссия 14

 Упорядочите стадии от простой настройки до полного режима строгой проверки:

1. Syntax only
2. Syntax + basic attributes
3. Semantic + custom rule overrides
4. Accessibility + policy rules
5. Strict mode (только исправления без изменений структуры)

Подсказка: Начинайте с простого синтаксиса, затем добавляйте слои проверки.

Ответ:

- Syntax only
- Syntax + basic attributes
- Semantic + custom rule overrides
- Accessibility + policy rules
- Strict mode (только исправления без изменений структуры)

Шаг 15:

Миссия 15

 Упорядочите уровни влияния на качество исправлений от самых внешних до самых глубоких:

1. Шаблон вывода (template)
2. Дополнительные правила / фильтры
3. Тип валидации (syntax / semantic / custom)
4. Параметры модели (temperature, top_p и др.)
5. Пост-обработка (postprocessor)

Подсказка: Внешние слои формируют общий каркас, внутренние — точность.

Ответ:

- Шаблон вывода (template)
- Дополнительные правила / фильтры
- Тип валидации (syntax / semantic / custom)
- Параметры модели (temperature, top_p и др.)
- Пост-обработка (postprocessor)

Шаг 16:

Миссия 16

 Упорядочите шаги тестирования конфигурации модели:

1. Запустить простую страницу с базовыми тегами
2. Добавить тег `img` без `alt` и проверить ошибку
3. Добавить вложенный тег нарушающего формата
4. Применить кастомные правила и проверить реакцию

5. Сравнить выводы с эталоном и скорректировать prompt

Подсказка: Начинайте с простого, постепенно добавляя сложность и проверяя результат.

Ответ:

- Запустить простую страницу с базовыми тегами
- Добавить тег `img` без `alt` и проверить ошибку
- Добавить вложенный тег нарушающего формата
- Применить кастомные правила и проверить реакцию
- Сравнить выводы с эталоном и скорректировать prompt

Шаг 17:

Миссия 17

✓ Финальное испытание: расположите уровни, влияющие на поведение модели от самого поверхностного до ядра:

1. Шаблон вывода (template)
2. Дополнительные правила / фильтры
3. Тип валидации
4. Параметры модели (temperature, max_tokens и др.)
5. Пост-обработка и финальные коррекции

Подсказка: Чем ближе к ядру, тем сильнее эффект на содержимое исправлений.

Ответ:

- Шаблон вывода (template)
- Дополнительные правила / фильтры
- Тип валидации
- Параметры модели (temperature, max_tokens и др.)
- Пост-обработка и финальные коррекции

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Алекс запускает LocalAI в режиме валидации HTML. Что он должен сделать первым?

Подсказка: настройка начинается с базовых параметров.

Ответ:

- **Указать схему правил HTML (DTD / custom)**
- Запустить модель без конфигурации
- Игнорировать ошибки на старте

Шаг 02:

Миссия 2

При тестировании HTML валидатор выдаёт предупреждения о пустых тегах. Что лучше сделать?

Подсказка: не все предупреждения равны — некоторые можно подавлять.

Ответ:

- **Добавить исключение для пустых тегов в конфигурации**

- Удалить все пустые теги вручную
- Игнорировать предупреждения глобально

🔗 Шаг 03:

Миссия 3

Мария хочет, чтобы валидатор автоматически фиксировал атрибуты без кавычек. Какой параметр ей нужен?

Подсказка: это касается синтаксического нормализатора.

Ответ:

- **enable_attribute_normalization**
- skip_unquoted_attrs
- auto_add_quotes

🔗 Шаг 04:

Миссия 4

Иван хочет, чтобы валидатор выдавал ошибку, если в HTML встречается inline-CSS. Что ему нужно запретить?

Подсказка: это касается стиля внутри тега.

Ответ:

- **inline_style_disallowed**
- forbid_css
- no_style_tag

🔗 Шаг 05:

Миссия 5

Валентин хочет, чтобы валидатор проверял `` только на наличие атрибута `alt`, но игнорировал другие предупреждения. Что выбрать?

Подсказка: настройка фильтрации предупреждений.

Ответ:

- **warning_filters = ['missing_alt']**
- error_only_imgs
- ignore_all_but_imgs

🔗 Шаг 06:

Миссия 6

Настройка LocalAI вывела к вам понятия и описания. Соотнесите параметр с его значением:

Параметры:

Ответ:

- strict_mode
- autofix_enabled
- error_threshold
- validation_schema

Значения:

1. Уровень строгости валидации (например, warn или error)

2. Автоматическое исправление ошибок
3. Схема, по которой проверяется HTML (например, HTML5 или DTD)
4. Количество ошибок, при превышении которых процесс падает

Подсказка: обдумайте, какой параметр отвечает за автоисправление, а какой — за схему.

- **strict_mode** — Уровень строгости валидации (например, warn или error)
- **autofix_enabled** — Автоматическое исправление ошибок
- **error_threshold** — Количество ошибок, при превышении которых процесс падает
- **validation_schema** — Схема, по которой проверяется HTML (например, HTML5 или DTD)

Шаг 07:

Миссия 7

Разработчики столкнулись с неожиданными терминами. Сопоставьте термин и его смысл:

Термины:

Ответ:

- False positive
- False negative
- Noise warning
- Silent fail

Определения:

1. Предупреждение, без критического значения
2. Ошибка не распознана валидатором
3. Предупреждение выдано ошибочно
4. Сбой без сообщения об ошибке

Подсказка: посмотрите на префиксы false / silent, подумайте, что значит «отсутствие».

- **False positive** — Предупреждение выдано ошибочно
- **False negative** — Ошибка не распознана валидатором
- **Noise warning** — Предупреждение, без критического значения
- **Silent fail** — Сбой без сообщения об ошибке

Шаг 08:

Миссия 8

При разборе конфигурации LocalAI часто встречаются сочетания. Сопоставьте фразу с её значением:

Фразы:

Ответ:

- “suppress_warnings”
- “repeatable_checks”
- “fallback_schema”
- “max_autofix_passes”

Определения:

1. Сколько раз попробовать автоисправление перед остановом
2. Какая схема используется при неудаче основной
3. Скрыть определённые типы предупреждений

4. Разрешить повторные проверки одного участка кода

Подсказка: `max` → максимум попыток, `fallback` → запасной вариант.

- **`suppress_warnings`** — Скрыть определённые типы предупреждений
- **`repeatable_checks`** — Разрешить повторные проверки одного участка кода
- **`fallback_schema`** — Какая схема используется при неудаче основной
- **`max_autofix_passes`** — Сколько раз пробовать автоисправление перед остановом

Шаг 09:

Миссия 9

Алексей сравнивает два режима: `sandbox` и `strict`. Сопоставьте режим с характерной чертой:

Режимы:

Ответ:

- `sandbox`
- `strict`
- `permissive`
- `custom`

Черты:

1. Минимальная валидация, только основные ошибки
2. Валидация по пользовательским правилам
3. Полная строгая проверка, ошибки как факты
4. Средний режим, с предупреждениями

Подсказка: `strict` → строгий, `permissive` → куда более мягкий, `sandbox` → базовый режим.

- **`sandbox`** — Минимальная валидация, только основные ошибки
- **`strict`** — Полная строгая проверка, ошибки как факты
- **`permissive`** — Средний режим, с предупреждениями
- **`custom`** — Валидация по пользовательским правилам

Шаг 10:

Миссия 10

Анна хочет понять последовательность этапов настройки модели валидации HTML.

Расположите шаги в правильном порядке:

1. Запуск проверки тестовым HTML-файлом
2. Определение схемы валидации
3. Настройка правил исправления и фильтров
4. Подключение конфигурации к LocalAI

Подсказка: сначала задаётся схема, затем правила, потом подключение, и лишь после — тестирование.

Ответ:

- Определение схемы валидации
- Настройка правил исправления и фильтров
- Подключение конфигурации к LocalAI
- Запуск проверки тестовым HTML-файлом

Шаг 11:

Миссия 11

При исправлении HTML модель должна пройти стадии. Расположите стадии по порядку:

1. Лексический разбор (tokenize)
2. Семантическая проверка (свойства тегов)
3. Синтаксический анализ (DOM-дерево)
4. Применение автокоррекции по правилам

Подсказка: сначала синтаксика, потом семантика, затем исправления.

Ответ:

- Лексический разбор (tokenize)
- Синтаксический анализ (DOM-дерево)
- Семантическая проверка (свойства тегов)
- Применение автокоррекции по правилам

Шаг 12:

Миссия 12

Последовательность реагирования на ошибки при пакете HTML-файлов:

1. Уведомление пользователя о найденных ошибках
2. Пороговая проверка количества ошибок (error_threshold)
3. Автоисправление с настройками
4. Прерывание процесса, если порог превышен

Подсказка: порог должен проверяться до исправлений, а прерывание — после проверки.

Ответ:

- Уведомление пользователя о найденных ошибках
- Пороговая проверка количества ошибок (error_threshold)
- Автоисправление с настройками
- Прерывание процесса, если порог превышен

Шаг 13:

Миссия 13

Расставьте этапы отладки конфига, если валидатор ругается:

1. Отключить все фильтры и проверить чистую схему
2. Подключить один фильтр и протестировать
3. Вернуться к конфигурации, если ошибка осталась
4. Добавлять фильтры по одному, контролируя результат

Подсказка: метод «разделяй и властвуй» помогает найти проблемный фильтр.

Ответ:

- Отключить все фильтры и проверить чистую схему
- Подключить один фильтр и протестировать
- Добавлять фильтры по одному, контролируя результат
- Вернуться к конфигурации, если ошибка осталась

Шаг 14:

Миссия 14

Последовательность повышения жёсткости валидации:

1. Permissive (мягкий режим)
2. Sandbox (базовый режим)
3. Custom (смешанный режим)
4. Strict (строгий режим)

Подсказка: подумайте, как меняется уровень контроля по мере роста жёсткости.

Ответ:

- Permissive (мягкий режим)
- Sandbox (базовый режим)
- Custom (смешанный режим)
- Strict (строгий режим)

Шаг 15:

Миссия 15

Составьте порядок действий при интеграции LocalAI в CI/CD:

1. Интеграция вызова валидатора в пайплайн
2. Добавление уведомлений о нарушениях
3. Настройка конфигурации HTML-валидации
4. Тестирование с репозиторием с ошибками

Подсказка: конфигурация должна быть готова до интеграции, тестирование — после встраивания.

Ответ:

- Настройка конфигурации HTML-валидации
- Интеграция вызова валидатора в пайплайн
- Тестирование с репозиторием с ошибками
- Добавление уведомлений о нарушениях

Шаг 16:

Миссия 16

При масштабировании проверки на множество HTML-страниц нужно соблюдать порядок:

1. Разделить страницы на группы
2. Запустить проверку группы с текущей конфигурацией
3. Собирать отчёты и агрегировать ошибки
4. Адаптировать фильтры по полученным результатам

Подсказка: сначала разбить, потом тестировать, потом анализировать, затем корректировать.

Ответ:

- Разделить страницы на группы
- Запустить проверку группы с текущей конфигурацией
- Собирать отчёты и агрегировать ошибки
- Адаптировать фильтры по полученным результатам

Шаг 17:

Миссия 17

Команда хочет ввести этап «обучение модели на ошибках пользователей». Расположите этапы в порядке внедрения:

1. Собирать реальные HTML-ошибки от пользователей
2. Обучить корректировочные правила на этих данных
3. Интегрировать новые правила в конфигурацию LocalAI
4. Мониторить, как изменяются предупреждения валидации

Подсказка: сначала сбор данных, затем обучение, внедрение, мониторинг.

Ответ:

- Собирать реальные HTML-ошибки от пользователей
- Обучить корректировочные правила на этих данных
- Интегрировать новые правила в конфигурацию LocalAI
- Мониторить, как изменяются предупреждения валидации

Модуль 4 - Модуль 4. Проверка HTML через внешние сервисы

Урок 1 - Обзор API W3C Validator

Урок 2 - Интеграция W3C и LocalAI

Урок 3 - Использование HTMLHint, Pa11y, Lighthouse

Урок 4 - Построение двойного потока LLM → W3C → LLM

Урок 5 - Обработка JSON-ответов от валидатора

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

 Ольга хочет проверить валидность HTML через внешний сервис W3C. С чего начать?

Подсказка: нужно отправить HTML на сервер автоматически.

Ответ:

- Отправить HTTP POST с HTML-контентом на API сервиса
- Сохранить HTML локально и ждать ответа вручную
- Запросить валидность по FTP
- открыть сайт сервиса и скопировать туда HTML

Шаг 02:

Миссия 2

Пётр интегрирует внешний валидатор. Какой формат ответа он ожидает?

Подсказка: обычно API возвращают структурированные ошибки.

Ответ:

- JSON с описанием ошибок и позициями

- Сырой HTML страницы
- Числовой код результата без данных
- XML-файл, вложенный в iframe

🔧 Шаг 03:

Миссия 3

🔗 Марина хочет, чтобы внешний валидатор не прекращал проверку при первой ошибке. Какой параметр установить?

Подсказка: часто API поддерживают опцию «continue on error».

Ответ:

- **continueOnError=true**
- stopOnFirstError=false
- failFast=false
- noAbort=true

🔧 Шаг 04:

Миссия 4

Иван хочет ограничить размер отправляемого HTML (например, 100 кБ). Что он должен сделать?

Подсказка: можно обрезать или отправлять лишь фрагмент.

Ответ:

- **Отправлять только первые 100 кБ или обрезанный фрагмент**
- Отключить проверку больших файлов
- Разбить HTML на много файлов
- Игнорировать ограничение сервиса

🔧 Шаг 05:

Миссия 5

🔒 Сергей хочет удостовериться, что связь с внешним сервисом защищена. Что он должен использовать?

Подсказка: передавайте HTML через безопасный протокол.

Ответ:

- **HTTPS / TLS при отправке запроса**
- FTP через несекуляр канал
- HTTP без шифрования
- SMTP отправка HTML

🔧 Шаг 06:

Миссия 6

Настройка API вернулась к вам. Соотнесите параметр API внешнего валидатора и его значение:

Параметры:

Ответ:

- api_key
- endpoint_url
- timeout_ms

- `max_retries`

Значения:

1. URL, куда посылаются запросы проверки
2. Ключ авторизации для доступа к API
3. Максимальное число попыток повтора при сбое
4. Таймаут ожидания ответа в миллисекундах

Подсказка: ключ — для авторизации, `endpoint` — адрес сервиса, `retries` — число попыток.

- **`api_key`** — Ключ авторизации для доступа к API
- **`endpoint_url`** — URL, куда посылаются запросы проверки
- **`timeout_ms`** — Таймаут ожидания ответа в миллисекундах
- **`max_retries`** — Максимальное число попыток повтора при сбое

Шаг 07:

Миссия 7

Валерий анализирует ответы API. Соотнесите статус ответа с его смыслом:

Статусы:

Ответ:

- 200 OK
- 400 Bad Request
- 401 Unauthorized
- 500 Internal Error

Смысл:

1. Запрос некорректен, синтаксическая ошибка
2. Токен API не пройден, нет доступа
3. **Успешно** — результат проверен
4. Сбой сервера на стороне сервиса

Подсказка: статус 200 — успех, 400 — запрос неверный, 401 — авторизация, 500 — внутренняя ошибка.

- **200 OK** — Успешно — результат проверен
- **400 Bad Request** — Запрос некорректен, синтаксическая ошибка
- **401 Unauthorized** — Токен API не пройден, нет доступа
- **500 Internal Error** — Сбой сервера на стороне сервиса

Шаг 08:

Миссия 8

Когда API возвращает JSON-ошибки, встречаются поля. Сопоставьте ключ и его содержание:

Ключи:

Ответ:

- `line`
- `column`
- `message`
- `severity`

Содержания:

1. Описание ошибки в текстовой форме
2. **Уровень** — warning или error
3. Номер строки, где ошибка найдена
4. Номер столбца в строке

Подсказка: line/column — позиция, message — текст, severity — степень.

- **line** — Номер строки, где ошибка найдена
- **column** — Номер столбца в строке
- **message** — Описание ошибки в текстовой форме
- **severity** — Уровень — warning или error

Шаг 09:

Миссия 9

Настя сравнивает виды интеграции внешнего валидатора. Сопоставьте способ и описание:

Способы:

Ответ:

- Ручной ввод через веб-интерфейс
- API REST
- Командная строка (CLI)
- Плагин редактора

Описание:

1. Интеграция прямо в IDE или редактор, подсветка ошибок сразу
2. Отправка запросов из кодовой логики через HTTP
3. Запуск валидатора в консоли с аргументами файла
4. Копировать и вставлять HTML вручную на сайте сервиса

Подсказка: API — через HTTP, CLI — командная строка, плагин — в редакторе.

- **Ручной ввод через веб-интерфейс** — Копировать и вставлять HTML вручную на сайте сервиса
- **API REST** — Отправка запросов из кодовой логики через HTTP
- **Командная строка (CLI)** — Запуск валидатора в консоли с аргументами файла
- **Плагин редактора** — Интеграция прямо в IDE или редактор, подсветка ошибок сразу

Шаг 10:

Миссия 10

Расположите этапы при обращении к API валидатора:

1. Формирование JSON-запроса с HTML
2. Отправка HTTP запроса
3. Получение ответа сервера
4. Парсинг JSON и обработка ошибок

Подсказка: сначала формируем запрос, потом отправляем, получаем ответ, потом анализируем.

Ответ:

- Формирование JSON-запроса с HTML
- Отправка HTTP запроса

- Получение ответа сервера
- Парсинг JSON и обработка ошибок

Шаг 11:

Миссия 11

При сбоях соединения с API необходима последовательность действий:

1. Проверить статус сети / DNS
2. Применить счастливый откат к локальному валидатору
3. Повторить запрос (retry)
4. Уведомить разработчика о сбое

Подсказка: сначала диагностика, потом fallback или retry, затем уведомление.

Ответ:

- Проверить статус сети / DNS
- Применить счастливый откат к локальному валидатору
- Повторить запрос (retry)
- Уведомить разработчика о сбое

Шаг 12:

Миссия 12

Когда API возвращает ошибки, как действовать:

1. Отфильтровать предупреждения по severity
2. Сортировать ошибки по файлу и позиции
3. Показать пользователю отчет
4. Сохранить лог ошибок

Подсказка: сначала фильтровать, затем сортировать, потом показать и зафиксировать.

Ответ:

- Отфильтровать предупреждения по severity
- Сортировать ошибки по файлу и позиции
- Показать пользователю отчет
- Сохранить лог ошибок

Шаг 13:

Миссия 13

При интеграции API в пайплайн выберите порядок:

1. Настроить API-конфигурацию
2. Добавить шаг проверки HTML через API
3. Обработать ошибки с отчётом
4. Остановить сборку при критических ошибках

Подсказка: конфигурация → вызов → обработка → остановка.

Ответ:

- Настроить API-конфигурацию
- Добавить шаг проверки HTML через API
- Обработать ошибки с отчётом

- Остановить сборку при критических ошибок

🔧 Шаг 14:

Миссия 14

При вводе fallback между внешним API и локальной проверкой:

1. Попытка проверки через внешний API
2. **При сбое** — использование локального валидатора
3. Возвращение результата пользователю
4. Логирование внешнего сбоя

Подсказка: сначала внешняя проверка, затем fallback, затем результат и лог.

Ответ:

- Попытка проверки через внешний API
- **При сбое** — использование локального валидатора
- Возвращение результата пользователю
- Логирование внешнего сбоя

🔧 Шаг 15:

Миссия 15

При масштабировании запросов к внешнему валидатору:

1. Ограничить число параллельных запросов
2. Распределять очередь запросов
3. Балансировать нагрузку на сервер API
4. Контролировать скорость отправки (rate limit)

Подсказка: нужно регулировать конкурентность, очередь и скорость.

Ответ:

- Ограничить число параллельных запросов
- Распределять очередь запросов
- Балансировать нагрузку на сервер API
- Контролировать скорость отправки (rate limit)

🔧 Шаг 16:

Миссия 16

При отказе API в определённые часы (maintenance window) действуйте так:

1. Обнаружить код ошибки 503 или 429
2. Переключиться на локальный валидатор временно
3. Повторять попытки через интервалы
4. Вернуться на внешний API при восстановлении

Подсказка: special статусы → fallback → retry → возврат.

Ответ:

- Обнаружить код ошибки 503 или 429
- Переключиться на локальный валидатор временно
- Повторять попытки через интервалы
- Вернуться на внешний API при восстановлении

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Лена внедряет проверку HTML и хочет использовать внешний валидатор. Как ей автоматически передать HTML-документ в сервис?

Подсказка: Передача должна быть в фоновом режиме, без ручного вмешательства.

Ответ:

- **Отправить HTTP POST с HTML в теле запроса**
- Вставить HTML вручную в браузере
- Передать HTML через SMTP-сервер
- Загрузить HTML на GitHub

Шаг 02:

Миссия 2

 Артём получает ответ от API-валидатора. Какой формат наиболее удобен для автоматической обработки?

Подсказка: Структурированный формат лучше всего подходит для разбора ошибок программно.

Ответ:

- **JSON с описанием ошибок**
- HTML-страница с отчётом
- PDF-файл
- Скриншот консоли

Шаг 03:

Миссия 3

 Света хочет получать отчёт обо всех ошибках, даже если их много. Какой параметр нужно включить в запросе?

Подсказка: Некоторые API позволяют не останавливаться на первой ошибке.

Ответ:

- **continueOnError=true**
- failImmediately=true
- verboseMode=false
- retryCount=0

Шаг 04:

Миссия 4

 Тимур отправляет большие HTML-файлы на проверку, но валидатор ограничивает размер. Как обойти ограничение?

Подсказка: Можно отправить только часть файла или его обрезанный вариант.

Ответ:

- **Обрезать HTML до лимита в 100 КБ**
- Увеличить лимит в настройках API
- Использовать FTP вместо API
- Разбить HTML на картинки

🔧 Шаг 05:

Миссия 5

🔒 Оля заботится о безопасности. Какой протокол использовать для надёжной передачи HTML на проверку?

Подсказка: Защита данных — обязательное условие при работе с внешними сервисами.

Ответ:

- **HTTPS (TLS)**
- HTTP
- FTP
- WS (WebSocket)

🔧 Шаг 06:

Миссия 6

🔑 Ваня подключает API. Что ему обязательно нужно передать в заголовке запроса?

Подсказка: Без этого вас могут не пустить в сервис.

Ответ:

- **API-ключ**
- UUID страницы
- URL клиента
- Путь к локальному HTML

🔧 Шаг 07:

Миссия 7

Антон интегрирует LocalAI с внешним валидатором. Сопоставьте понятия и их описания.

Подсказка: Внимательно посмотрите на роли компонентов API.

Ответ:

- **Access Token** — Ключ, дающий доступ к API
- **Endpoint** — URL-адрес сервиса, к которому обращается клиент
- **Payload** — Содержимое, передаваемое в теле запроса
- **Response** — Ответ, который возвращает API после запроса

🔧 Шаг 08:

Миссия 8

Катя замечает, что LocalAI не всегда правильно понимает ошибки в HTML. Сопоставьте типы ошибок с их объяснением.

Подсказка: Ошибки бывают как синтаксическими, так и логическими.

Ответ:

- **Missing closing tag** — Открыт тег, но не закрыт
- **Invalid nesting** — Один тег вложен в другой некорректно
- **Deprecated tag** — Использован устаревший HTML-тег
- **Attribute without value** — Атрибут указан, но без значения

🔧 Шаг 09:

Миссия 9

📖 Макс хочет автоматизировать разбор ответа от валидатора. Найди соответствие между полями JSON и их содержанием.

Подсказка: Структура ответа API может быть разной, но поля часто стандартизированы.

Ответ:

- **line** — Номер строки, где произошла ошибка
- **message** — Описание ошибки
- **type** — Категория ошибки
- **extract** — Фрагмент HTML с ошибкой

🔮 Шаг 10:

Миссия 10

👤 Мила написала скрипт для проверки HTML через API. Сопоставь шаги работы скрипта и действия.

Подсказка: Обрати внимание на стандартный жизненный цикл запроса.

Ответ:

- **1. Подготовка HTML** — Загрузка HTML-файла или генерация содержимого
- **2. Отправка запроса** — POST-запрос с телом HTML
- **3. Получение ответа** — Получение JSON с результатами
- **4. Обработка данных** — Анализ ошибок и их отображение

🔮 Шаг 11:

Миссия 11

✂ Дима хочет различать типы валидации. Сопоставь тип валидации и её назначение.

Подсказка: Некоторые проверки ориентированы на структуру, другие — на доступность.

Ответ:

- **Structural validation** — Проверка правильности тегов и их вложенности
- **Accessibility validation** — Анализ доступности для людей с ограничениями
- **SEO validation** — Проверка соответствия требованиям поисковых систем
- **Security validation** — Проверка на XSS и небезопасные элементы

🔮 Шаг 12:

Миссия 12

🔗 Расставь шаги автоматической проверки HTML через внешний сервис.

Подсказка: Вспомни, как строится API-поток: от запроса до вывода результата.

Ответ:

- Подготовка HTML-файла
- Отправка POST-запроса на API
- Получение JSON-ответа
- Парсинг результата и вывод ошибок

🔮 Шаг 13:

Миссия 13

📋 Представим, что LocalAI анализирует ошибки. Расставь их по приоритету устранения — от самых критичных к менее важным.

Подсказка: Ошибки, нарушающие структуру, исправляются в первую очередь.

Ответ:

- Незакрытый тег
- Неправильная вложенность тегов
- Отсутствие alt у изображения
- Использование устаревшего тега

Модуль 5 - Модуль 5. API и интеграции

Урок 1 - Работа с LocalAI API в Python

Урок 2 - Использование API в Go

Урок 3 - Создание REST API для HTML-валидации

Урок 4 - Встраивание в веб-интерфейс (Flask или GoFiber)

Урок 5 - Сборка Docker-compose окружения

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Вы — разработчик, создаёте API-эндпоинт для проверки HTML-разметки. Какой метод HTTP вы выберете для операции проверки (без изменения данных)?

Подсказка: Подумайте, что означает «без изменения данных» в REST.

Ответ:

- GET
- POST
- PUT

Шаг 02:

Миссия 2

 Вы хотите передать HTML-код тела страницы через API. Какой формат передачи предпочтителен?

Подсказка: JSON — удобный формат для REST-запросов.

Ответ:

- JSON с полем html
- HTML напрямую в URL
- XML с CDATA

Шаг 03:

Миссия 3

 Вы добавили поддержку проверки через API: клиент отправляет запрос, сервер возвращает JSON с ошибками. Как обозначить успешный ответ, когда ошибок нет?

Подсказка: Подумайте о кодах HTTP и общее их значение.

Ответ:

- HTTP 200 OK с пустым массивом ошибок

- HTTP 500 Internal Server Error
- HTTP 204 No Content

🔧 Шаг 04:

Миссия 4

🔧 Разработчик хочет, чтобы API позволял отправлять как одиночный HTML, так и множество документов (bulk). Какой подход REST-архитектуры лучше?

Подсказка: Подумайте, можно ли передать массив объектов в JSON.

Ответ:

- **Принимать JSON-массив объектов {html:...}**
- Разделять через запятую в строке
- Отправлять через multipart/form-data с файлами

🔧 Шаг 05:

Миссия 5

✂ Вы хотите ограничить доступ к API проверки HTML. Какой способ аутентификации подходит лучше всего?

Подсказка: Подумайте, как защитить открытое API от злоупотреблений.

Ответ:

- **API ключ (token) в заголовке Authorization**
- Без авторизации
- Basic auth с логином/паролем в URL

🔧 Шаг 06:

Миссия 6

📦 Вы хотите, чтобы API мог возвращать не только список ошибок, но и предложения по исправлению. Как логично организовать это в JSON-ответе?

Подсказка: Подумайте о структуре: ошибка + предложение — вместе.

Ответ:

- **Массив объектов с полями error и suggestion**
- Строка с текстом ошибок и предложений
- Два отдельные массива ошибок и предложений

🔧 Шаг 07:

Миссия 7

🔗 Сопоставьте термин API-интеграции с его описанием в контексте LocalAI проверки HTML.

Термины:

Ответ:

- Webhook
- Rate limit
- Endpoint
- Payload

Определения:

1. URL, по которому клиент делает запрос к API

2. Объём данных, отправляемых в запросе
3. Ограничение числа запросов за единицу времени
4. Механизм уведомления клиента при событии

Подсказка: Проанализируйте, когда используется webhook и как он отличается от обычного вызова.

- **Webhook** — Механизм уведомления клиента при событии
- **Rate limit** — Ограничение числа запросов за единицу времени
- **Endpoint** — URL, по которому клиент делает запрос к API
- **Payload** — Объём данных, отправляемых в запросе

Шаг 08:

Миссия 8

 Сопоставьте ключевые поля JSON-ответа от API LocalAI к их смыслу.

Поля:

Ответ:

- status
- errors
- suggestion
- metadata

Значения:

1. Список найденных ошибок в разметке
2. Информация о проверке (время, версия и т.д.)
3. Код ответа (ок/ошибки)
4. Предложение исправления для ошибки

Подсказка: Подумайте: статус — это код ответа, metadata — технические данные.

- **status** — Код ответа (ок/ошибки)
- **errors** — Список найденных ошибок в разметке
- **suggestion** — Предложение исправления для ошибки
- **metadata** — Информация о проверке (время, версия и т.д.)

Шаг 09:

Миссия 9

 Сопоставьте шаги взаимодействия двух сервисов (клиент ↔ LocalAI API) с описаниями.

Шаги:

Ответ:

- Клиент отправляет запрос
- LocalAI обрабатывает HTML
- LocalAI возвращает JSON
- Клиент парсит ответ

Описания:

1. Анализ ошибок и генерация предложений
2. Получение данных о результатах (status, errors...)
3. Отправка HTML на проверку

4. Интеграция предложений в редактор

Подсказка: Сопоставьте логичный поток: запрос → обработка → ответ → использование.

- **Клиент отправляет запрос** — Отправка HTML на проверку
- **LocalAI обрабатывает HTML** — Анализ ошибок и генерация предложений
- **LocalAI возвращает JSON** — Получение данных о результатах (status, errors...)
- **Клиент парсит ответ** — Интеграция предложений в редактор

🔮 Шаг 10:

Миссия 10

♥ При интеграции API важны заголовки. Сопоставьте заголовок с его назначением.

Заголовки:

Ответ:

- Content-Type
- Authorization
- X-RateLimit-Remaining
- Accept

Назначения:

1. Сколько запросов осталось до лимита
2. Тип отправляемых данных (например application/json)
3. Требуемый формат ответа
4. Токен доступа в API

Подсказка: Authorization — про безопасность, Accept — про формат ответа.

- **Content-Type** — Тип отправляемых данных (например application/json)
- **Authorization** — Токен доступа в API
- **X-RateLimit-Remaining** — Сколько запросов осталось до лимита
- **Accept** — Требуемый формат ответа

🔮 Шаг 11:

Миссия 11

📋 Упорядочите этапы создания интеграции LocalAI API с редактором HTML:

1. Написание кода редактора, который вызывает API
2. Обработка JSON-ответа и применение исправлений
3. Настройка API-ключа и заголовков
4. Отправка HTML-кода на API
5. Тестирование функционирования

Подсказка: Какой последовательности логически придерживаться, от настроек до тестов?

Ответ:

- Настройка API-ключа и заголовков
- Написание кода редактора, который вызывает API
- Отправка HTML-кода на API
- Обработка JSON-ответа и применение исправлений
- Тестирование функционирования

Шаг 12:

Миссия 12

📋 Упорядочите уровни ответа API в порядке от самой общей информации к детальной:

1. metadata
2. status
3. errors
4. fix suggestions внутри errors

Подсказка: Сначала общее, потом детали, потом вложенное.

Ответ:

- metadata
- status
- errors
- fix suggestions внутри errors

Шаг 13:

Миссия 13

🔄 Упорядочите шаги обработки одного HTML-документа на сервере LocalAI:

1. Парсинг DOM
2. Сбор списка тегов и атрибутов
3. Проверка на правила (например, закрыты ли теги)
4. Формирование списка ошибок и предложений
5. Сериализация JSON и отправка ответа

Подсказка: Подумайте, как проходит работа движка: сначала анализ, затем проверка, потом ответ.

Ответ:

- Парсинг DOM
- Сбор списка тегов и атрибутов
- Проверка на правила (например, закрыты ли теги)
- Формирование списка ошибок и предложений
- Сериализация JSON и отправка ответа

Шаг 14:

Миссия 14

⚡ Упорядочите стадии тестирования API-решения LocalAI:

1. Проверка обработки валидного HTML
2. Проверка обработки пустого тела
3. Проверка обработки HTML с ошибками
4. Проверка предельного размера тела
5. Проверка устойчивости при множественных параллельных запросах

Подсказка: Тестируйте от простого к сложному, затем нагрузку.

Ответ:

- Проверка обработки валидного HTML
- Проверка обработки пустого тела

- Проверка обработки HTML с ошибками
- Проверка предельного размера тела
- Проверка устойчивости при множественных параллельных запросах

🔧 Шаг 15:

Миссия 15

📋 Упорядочите реакции сервера на разные классы ошибок API:

1. **400 Bad Request** — некорректный JSON или синтаксическая ошибка
2. **401 Unauthorized** — отсутствует или неправильный API-ключ
3. **422 Unprocessable Entity** — HTML-разметка не может быть распознана
4. **429 Too Many Requests** — превышен rate-limit
5. **500 Internal Server Error** — неожиданный сбой

Подсказка: Расставьте по возрастанию от клиента к серверу по типу ошибки.

Ответ:

- **400 Bad Request** — некорректный JSON или синтаксическая ошибка
- **401 Unauthorized** — отсутствует или неправильный API-ключ
- **422 Unprocessable Entity** — HTML-разметка не может быть распознана
- **429 Too Many Requests** — превышен rate-limit
- **500 Internal Server Error** — неожиданный сбой

🔧 Шаг 16:

Миссия 16

★ Упорядочите уровни интеграции LocalAI API в продукте от лёгкого к глубокому:

1. Кнопка «Проверить HTML» вручную
2. Автоматическая проверка при каждом сохранении документа
3. Фоновые проверки при вводе (live)
4. Интеграция с IDE / редактором с подсветкой ошибок
5. CI/CD-пайплайн проверок на каждом коммите

Подсказка: От простого ручного до полного автоматического в pipeline.

Ответ:

- Кнопка «Проверить HTML» вручную
- Автоматическая проверка при каждом сохранении документа
- Фоновые проверки при вводе (live)
- Интеграция с IDE / редактором с подсветкой ошибок
- CI/CD-пайплайн проверок на каждом коммите

📖 Урок 7 - Викторина 2

🔧 Шаг 01:

Миссия 1

Вы только что развернули сервер LocalAI, и хотите убедиться, что API доступен. Какой HTTP-метод вы используете для «ping»-запроса (без передачи данных)?

Подсказка: Такой запрос должен быть «без побочных эффектов».

Ответ:

- GET
- POST
- PUT

Шаг 02:

Миссия 2

 Клиент хочет отправить HTML-контент на проверку. Где логичнее поместить его в HTTP-запросе?

Подсказка: Тело запроса — место для данных, а не URL.

Ответ:

- **В теле запроса (request body) как JSON поле html**
- В URL как параметр ?html=...
- В HTTP-заголовке

Шаг 03:

Миссия 3

 Вы хотите ограничить доступ к API проверки. Что выбрать?

Подсказка: Без защиты API станет уязвимым.

Ответ:

- **API-ключ (токен) в заголовке Authorization**
- Без авторизации
- Авторизация через URL параметры

Шаг 04:

Миссия 4

 Вы решили возвращать не только ошибки, но и советы по исправлению. Как лучше структурировать ответ?

Подсказка: Логично связать ошибку и её исправление вместе.

Ответ:

- **Массив объектов {error, suggestion}**
- Один общий текст ошибки и предложения
- Два параллельных массива ошибок и предложений

Шаг 05:

Миссия 5

 Сопоставьте элемент API-интеграции с его ролью в LocalAI:

Элементы:

Ответ:

- Rate limit
- Timeout
- Webhook
- Callback URL

Роли:

1. Максимальное число запросов за единицу времени
2. Максимальное время ожидания ответа
3. URL, вызываемый автоматически при событии
4. Адрес для получения уведомления о завершении проверки

Подсказка: Timeout относится к задержке, webhook к уведомлению.

- **Rate limit** — Максимальное число запросов за единицу времени
- **Timeout** — Максимальное время ожидания ответа
- **Webhook** — URL, вызываемый автоматически при событии
- **Callback URL** — Адрес для получения уведомления о завершении проверки

Шаг 06:

Миссия 6

🔗 Сопоставьте поля JSON-ответа LocalAI с их значением:

Поля:

Ответ:

- status
- errors
- fixes
- info

Значения:

1. общий результат (ok / fail)
2. список найденных ошибок
3. список предложений исправления
4. технические сведения (время, версия и т.д.)

Подсказка: Поле info часто используется для метаданных.

- **status** — общий результат (ok / fail)
- **errors** — список найденных ошибок
- **fixes** — список предложений исправления
- **info** — технические сведения (время, версия и т.д.)

Шаг 07:

Миссия 7

🔗 Сопоставьте заголовок HTTP и его роль в API-ответе:

Заголовки:

Ответ:

- Content-Type
- Authorization
- X-RateLimit-Limit
- X-RateLimit-Reset

Роли:

1. формат тела ответа (например, JSON)

2. ключ доступа клиента
3. максимальное количество запросов за период
4. время, когда лимит сбросится

Подсказка: X-RateLimit-* часто показывает лимиты.

- **Content-Type** — формат тела ответа (например, JSON)
- **Authorization** — ключ доступа клиента
- **X-RateLimit-Limit** — максимальное количество запросов за период
- **X-RateLimit-Reset** — время, когда лимит сбросится

Шаг 08:

Миссия 8

 Упорядочите шаги вызова API клиентом:

1. Сбор HTML-кода страницы
2. Добавление заголовков (Authorization, Content-Type)
3. Формирование JSON с ключом html
4. Отправка HTTP-запроса на endpoint
5. Ожидание и получение JSON-ответа
6. Обработка ответа

Подсказка: От подготовки данных к отправке, затем к получению и обработке.

Ответ:

- Сбор HTML-кода страницы
- Добавление заголовков (Authorization, Content-Type)
- Формирование JSON с ключом html
- Отправка HTTP-запроса на endpoint
- Ожидание и получение JSON-ответа
- Обработка ответа

Шаг 09:

Миссия 9

 Упорядочите уровни интеграции API в редакторе:

1. Кнопка «Проверить» вручную
2. Проверка при сохранении
3. Live-проверка при вводе
4. Подсветка ошибок внутри редактора
5. Проверка на этапе CI/CD

Подсказка: От ручного к автоматизации.

Ответ:

- Кнопка «Проверить» вручную
- Проверка при сохранении
- Live-проверка при вводе
- Подсветка ошибок внутри редактора
- Проверка на этапе CI/CD

🔧 Шаг 10:

Миссия 10

Упорядочите проверки API по сложности:

1. пустой HTML
2. валидный HTML без ошибок
3. HTML с одной ошибкой
4. HTML с множеством ошибок
5. максимально длинный HTML

Подсказка: Тестируйте от простого к самому сложному.

Ответ:

- пустой HTML
- валидный HTML без ошибок
- HTML с одной ошибкой
- HTML с множеством ошибок
- максимально длинный HTML

🔧 Шаг 11:

Миссия 11

🔄 Упорядочите стадии обработки HTML на сервере:

1. парсинг HTML в DOM или AST
2. применение правил проверки (валидация тегов/атрибутов)
3. генерация списка ошибок и рекомендаций
4. формирование JSON-ответа
5. отправка ответа клиенту

Подсказка: Это внутренний pipeline обработки.

Ответ:

- парсинг HTML в DOM или AST
- применение правил проверки (валидация тегов/атрибутов)
- генерация списка ошибок и рекомендаций
- формирование JSON-ответа
- отправка ответа клиенту

🔧 Шаг 12:

Миссия 12

🐞 Упорядочите реакции API на типичные ошибки клиента:

1. **400 Bad Request** — некорректный JSON
2. **401 Unauthorized** — отсутствует токен
3. **422 Unprocessable Entity** — HTML не распознаётся
4. **429 Too Many Requests** — превысили лимит
5. **500 Internal Server Error** — сбой на сервере

Подсказка: От ошибок клиента к сбоям сервера.

Ответ:

- **400 Bad Request** — некорректный JSON

- **401 Unauthorized** — отсутствует токен
- **422 Unprocessable Entity** — HTML не распознаётся
- **429 Too Many Requests** — превысили лимит
- **500 Internal Server Error** — сбой на сервере

Шаг 13:

Миссия 13

 Упорядочите уровни интеграции LocalAI в продукте от простого к продвинутому:

1. кнопка «Проверить HTML»
2. проверка при сохранении документа
3. live-проверка во время набора текста
4. интеграция с редактором (подсветка ошибок)
5. встроенная проверка в CI/CD

Подсказка: От внешнего действия к полной автоматизации.

Ответ:

- кнопка «Проверить HTML»
- проверка при сохранении документа
- live-проверка во время набора текста
- интеграция с редактором (подсветка ошибок)
- встроенная проверка в CI/CD

Шаг 14:

Миссия 14

 Упорядочите фазы тестирования API по возрастанию сложности:

1. тест с валидным HTML
2. тест с одной ошибкой
3. тест с множеством ошибок
4. тест с пустым содержимым
5. нагрузочное тестирование (много параллельных запросов)

Подсказка: Сначала лёгкие случаи, затем граничные и нагрузочные.

Ответ:

- тест с валидным HTML
- тест с одной ошибкой
- тест с множеством ошибок
- тест с пустым содержимым
- нагрузочное тестирование (много параллельных запросов)

Модуль 6 - Модуль 6. Создание веб-интерфейса

 Урок 1 - Создание интерфейса с HTML и Tailwind

 Урок 2 - Отображение ошибок и подсветка diff

 Урок 3 - Валидация в реальном времени через Fetch API

 Урок 4 - Настройка backend-frontend связи

 Урок 5 - Развёртывание локально и в сети

 Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Веб-интерфейс загрузки HTML: Петя добавил поле `<input type="file">`, но кнопка «Загрузить» не активируется. В чём может быть проблема?

Подсказка: Проверь, установлен ли атрибут ассерт или слушатель события.

Ответ:

- Нет обработчика события `change` или `click`
- Файл слишком большой для поля `input`
- Неправиль MIME-тип в атрибуте `accept`

Шаг 02:

Миссия 2

Лена создала кнопку «Проверить HTML» и хочет, чтобы при клике отправлялось содержимое `textarea` на локальный сервер. Какой метод JavaScript наиболее подходящий?

Подсказка: Подумай, нужен ли ответ от сервера.

Ответ:

- **fetch с методом POST**
- `window.location = URL`
- `document.write`

Шаг 03:

Миссия 3

Вася хочет предотвратить перезагрузку страницы при отправке формы проверки. Что ему делать?

Подсказка: Событие `onsubmit` и стандартное поведение формы.

Ответ:

- **`event.preventDefault()` в обработчике `onsubmit`**
- `return false` в конце скрипта
- Удалить атрибут `action` у формы

Шаг 04:

Миссия 4

Марина вставляет элемент `<div id="output">`, где хочет отобразить результаты проверки. Какой способ безопаснее вставки HTML-ответа с сервера?

Подсказка: Подумай о XSS и innerHTML.

Ответ:

- **textContent** или **createElement + append**
- innerHTML напрямую вставлять весь текст
- document.write ответа

🔮 Шаг 05:

Миссия 5

Нина затемняет фон викторины серой полупрозрачной маской `<div class="overlay">`. Но при клике по маске всё равно срабатывают элементы под ней. Что не хватает?

Подсказка: Подумай об указателе событий и о том, блокирует ли элемент события мыши.

Ответ:

- **pointer-events: all** и **слушатель click на overlay**
- `opacity: 0.5`
- `position: absolute`

🔮 Шаг 06:

Миссия 6

Когда Лёша строит интерфейс, он сталкивается с терминами. Сопоставьте термин и его значение:

Термины:

Ответ:

- DOM
- CSSOM
- shadow DOM
- innerText

Значения:

1. Представление стилей документа как дерева объектов
2. Специальная изолированная часть дерева DOM для Web Components
3. Объектная модель документа (дерево узлов)
4. Текстовое содержимое элемента, игнорируя скрытые теги

Подсказка: Подумай, что DOM — это документ, CSSOM — стили, а shadow DOM изолирует компоненты.

- **DOM** — Объектная модель документа (дерево узлов)
- **CSSOM** — Представление стилей документа как дерева объектов
- **shadow DOM** — Специальная изолированная часть дерева DOM для Web Components
- **innerText** — Текстовое содержимое элемента, игнорируя скрытые теги

🔮 Шаг 07:

Миссия 7

В интерфейсе проверки всплывают ошибки разного типа. Сопоставьте код ошибки и её значение:

Коды:

Ответ:

- 400
- 404
- 500
- 422

Значения:

1. Синтаксическая ошибка в запросе или неверные параметры
2. Не удалось найти ресурс (например, API проверки не существует)
3. Внутренняя ошибка сервера при проверке
4. Неправильный формат данных (валидатор не понял данные)

Подсказка: 400 — общая ошибка запроса, 422 — часто формат неверен.

- **400** — Синтаксическая ошибка в запросе или неверные параметры
- **404** — Не удалось найти ресурс (например, API проверки не существует)
- **500** — Внутренняя ошибка сервера при проверке
- **422** — Неправильный формат данных (валидатор не понял данные)

Шаг 08:

Миссия 8

В оформлении панели ошибок используются термины. Сопоставьте термин и его роль:

Термины:

Ответ:

- Toast
- Modal
- Inline error
- Tooltip

Роли:

1. Маленькое сообщение поверх интерфейса, исчезающее само
2. Оверлейное окно поверх всего интерфейса
3. Ошибка отображается рядом с полем ввода
4. Подсказка при наведении курсора

Подсказка: Подумай, где обычно показывают сообщение «сохранено» или подсказки при hover.

- **Toast** — Маленькое сообщение поверх интерфейса, исчезающее само
- **Modal** — Оверлейное окно поверх всего интерфейса
- **Inline error** — Ошибка отображается рядом с полем ввода
- **Tooltip** — Подсказка при наведении курсора

Шаг 09:

Миссия 9

При разработке интерфейса, Василиса рассматривает разные события. Сопоставьте событие и его поведение:

События:

Ответ:

- DOMContentLoaded
- load
- click
- input

Поведение:

1. Срабатывает, когда весь документ и ресурсы загружены
2. Срабатывает, когда только DOM готов, но не ресурсы
3. При каждом вводе текста в поле формы
4. При нажатии на элемент

Подсказка: Помни, DOMContentLoaded наступает раньше load.

- **DOMContentLoaded** — Срабатывает, когда только DOM готов, но не ресурсы
- **load** — Срабатывает, когда весь документ и ресурсы загружены
- **click** — При нажатии на элемент
- **input** — При каждом вводе текста в поле формы

 Шаг 10:

Миссия 10

Упорядочьте шаги отправки HTML из интерфейса на проверку:

1. Получить ответ сервера
2. Подготовить JSON с содержимым HTML
3. Обработать результат и показать пользователю
4. Отправить запрос методом POST
5. Ожидание ответа (await / .then)

Подсказка: Подумай, что сначала готовим данные, затем отправляем и ждём, затем обрабатываем.

Ответ:

- Подготовить JSON с содержимым HTML
- Отправить запрос методом POST
- Ожидание ответа (await / .then)
- Получить ответ сервера
- Обработать результат и показать пользователю

 Шаг 11:

Миссия 11

У arrange последовательность этапов отображения ошибок пользователю:

1. Скрыть предыдущие уведомления
2. Добавить новый элемент ошибки в DOM
3. Применить стили и анимацию показа
4. Удалить уведомление через таймаут
5. Прослушка события «закрыть» (кнопка x)

Подсказка: Подумай, что сначала скрываешь старое, показываешь новое, затем ставишь таймер.

Ответ:

- Скрыть предыдущие уведомления
- Добавить новый элемент ошибки в DOM
- Применить стили и анимацию показа
- Прослушка события «закрыть» (кнопка x)
- Удалить уведомление через таймаут

 Шаг 12:

Миссия 12

Упорядочьте этапы жизненного цикла компонента веб-интерфейса (например, Vue / Web Component):

1. mount / подключение к DOM
2. рендеринг в DOM
3. обновление (при изменении данных)
4. демонтаж / удаление из DOM
5. инициализация состояния

Подсказка: Инициализация всегда первична, а демонтаж — финален.

Ответ:

- инициализация состояния
- рендеринг в DOM
- обновление (при изменении данных)
- mount / подключение к DOM
- демонтаж / удаление из DOM

 Шаг 13:

Миссия 13

Упорядочьте слои интерфейса перед отправкой данных на сервер:

1. Слой представления (View)
2. Слой данных (Model / state)
3. Слой логики / контроллера (Controller)
4. Слой сетевого запроса (API)
5. Слой отображения результата (Feedback)

Подсказка: Данные проходят от View → Model → логика → API → отображение.

Ответ:

- Слой представления (View)
- Слой данных (Model / state)
- Слой логики / контроллера (Controller)
- Слой сетевого запроса (API)
- Слой отображения результата (Feedback)

 Шаг 14:

Миссия 14

Юра на фронтенде использует кнопку «Проверить» и хочет дать визуальный отклик — сделать спиннер или «Проверка...». Как лучше показывать это состояние?

Подсказка: Подумай о состоянии загрузки в React или vanilla JS.

Ответ:

- **Добавить класс loading и показывать спиннер внутри кнопки**
- Блокировать весь экран div overlay на время проверки
- **Ничего не делать** — ждать silently

Шаг 15:

Миссия 15

Упорядочите шаги обработки ошибок HTTP для пользователя:

1. Проверить статус код ответа
2. Парсить тело ответа (JSON или текст)
3. Определить сообщение для пользователя
4. Показать сообщение в интерфейсе
5. Логирование на сервер или консоль

Подсказка: После получения ответа определяется, что дальше делать: показывать или логировать.

Ответ:

- Проверить статус код ответа
- Парсить тело ответа (JSON или текст)
- Определить сообщение для пользователя
- Показать сообщение в интерфейсе
- Логирование на сервер или консоль

Шаг 16:

Миссия 16

Упорядочите этапы при получении и визуализации списка ошибок HTML:

1. Очистить контейнер для ошибок
2. Для каждой ошибки создать элемент li / div
3. Установить текст и стиль ошибки
4. Добавить элемент в список (ul или div)
5. Прокрутить к первому элементу ошибки по месту

Подсказка: Сначала очистить старое, затем создать новое, затем прокрутка.

Ответ:

- Очистить контейнер для ошибок
- Для каждой ошибки создать элемент li / div
- Установить текст и стиль ошибки
- Добавить элемент в списка (ul или div)
- Прокрутить к первому элементу ошибки по месту

Шаг 17:

Миссия 17

Финальная задача: Игорь хочет, чтобы после исправления HTML на сервере интерфейс автоматически обновил исходный textarea без перезагрузки. Что ему использовать?

Подсказка: Подумай, как обновить содержимое элемента после ответа.

Ответ:

- `textarea.value = исправленный HTML из ответа`
- `window.reload()`
- `alert(ответ)` и копировать вручную

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Вы создаёте веб-интерфейс для локального HTML-валидатора. Какой элемент HTML вы бы использовали, чтобы показать пользователю выбор из нескольких вариантов?

Подсказка: Это стандартный элемент формы, который раскрывается.

Ответ:

- **элемент**
- элемент `</li>` `<li><fieldset>` элемент `</li>` `</ul></details>`

Шаг 02:

Миссия 2

✎ В интерфейсе вы даёте пользователю кнопку «Проверить». Какой атрибут нужно указать, чтобы при нажатии на неё отправить форму без перезагрузки страницы?

Подсказка: Используется атрибут, который связывает форму с JavaScript.

Ответ:

- **onclick**
- href
- action

Шаг 03:

Миссия 3

🗉 Вы хотите отобразить результаты проверки в специальном контейнере под формой. Какой подход лучше использовать?

Подсказка: Современный способ — менять содержимое динамически.

Ответ:

- **innerHTML или textContent в JS**
- `window.open()`
- `alert()` окно

Шаг 04:

Миссия 4

✎ Вы хотите, чтобы при вводе HTML и нажатии кнопки сразу запускалась проверка. Какой слушатель события применить к форме?

Подсказка: Используют событие отправки формы, чтобы предотвратить дефолтное действие.

Ответ:

- **onsubmit**
- onchange
- onclick

🔗 Шаг 05:

Миссия 5

🔗 Сопоставьте сценарии с обработчиками событий в вашем интерфейсе LocalAI.

Сценарии:

Ответ:

- Пользователь нажал «Проверить»
- Пользователь изменил текст в `textarea`
- Пользователь выбрал другой режим в `select`
- Проход завершился, готовы результаты

Обработчики:

1. `onchange` для `select`
2. `onsubmit` для формы
3. `input` для `textarea`
4. функция `callback` после завершения анализа

Подсказка: Запуск анализа — `callback`, выбор режима — `onchange`.

- **Пользователь нажал «Проверить»** — `onsubmit` для формы
- **Пользователь изменил текст в `textarea`** — `input` для `textarea`
- **Пользователь выбрал другой режим в `select`** — `onchange` для `select`
- **Проход завершился, готовы результаты** — функция `callback` после завершения анализа

🔗 Шаг 06:

Миссия 6

🔗 Какой режим в интерфейсе можно предложить пользователю, чтобы варьировать строгость проверки HTML?

Подсказка: Подумайте о режимах «жёсткий» и «мягкий».

Ответ:

- **Soft / Strict режимы**
- Light / Dark темы
- Debug / Release режимы

🔗 Шаг 07:

Миссия 7

🔗 Вы решили визуально подсветить ошибки в HTML-коде на стороне клиента. Какой приём использовать?

Подсказка: Можно оборачивать проблемные фрагменты в специальный тег с классом.

Ответ:

- **Оборачивать ошибочные фрагменты в `span` с классом `error`**
- Вставлять отдельные блоки `canvas`
- Перерисовывать весь HTML как изображение

🔗 Шаг 08:

Миссия 8

🔗 Сопоставьте элементы интерфейса с CSS-классами, которые вы могли бы применить:

Элементы:

Ответ:

- Ошибка текста в span
- Бокс с результатами
- Кнопка «Проверить»
- Textarea ввода

CSS-классы:

1. .highlight-error
2. .results-container
3. .check-button
4. .input-area

Подсказка: Названия классов часто отражают роль элемента.

- **Ошибка текста в span** — .highlight-error
- **Бокс с результатами** — .results-container
- **Кнопка «Проверить»** — .check-button
- **Textarea ввода** — .input-area

🔗 Шаг 09:

Миссия 9

🔗 Сопоставьте методы JavaScript с их применением в веб-интерфейсе LocalAI:

Методы:

Ответ:

- fetch()
- addEventListener()
- preventDefault()
- querySelector()

Применения:

1. Отключение стандартного submit формы
2. Поиск элемента контейнера
3. Отправка запроса к бэкенду проверки
4. Назначение обработчика на кнопку или форма

Подсказка: fetch — запрос, preventDefault — отмена действия.

- **fetch()** — Отправка запроса к бэкенду проверки
- **addEventListener()** — Назначение обработчика на кнопку или форма
- **preventDefault()** — Отключение стандартного submit формы
- **querySelector()** — Поиск элемента контейнера

🔗 Шаг 10:

Миссия 10

🔗 Какой формат данных выбрать между фронтендом и LocalAI-сервисом, чтобы передавать результаты проверки?

Подсказка: Это популярный текстовый формат для структурированных данных.

Ответ:

- **JSON**
- XML
- Plain text

 **Шаг 11:**

Миссия 11

△ Вы обнаружили, что пользователь вводит некорректный CSS в HTML. Какой режим проверки лучше включить?

Подсказка: Нужно определить, разрешается ли CSS внутри style-атрибутов.

Ответ:

- **Strict HTML + CSS режим**
- Lenient HTML-only режим
- No-check режим

 **Шаг 12:**

Миссия 12

🔍 Расположите шаги, которые интерфейс LocalAI выполняет при проверке HTML:

1. Отправить HTML-код и параметры на сервер
2. Установить слушатель события submit
3. Отобразить результаты пользователю
4. Получить ответ от сервера
5. Препятствовать стандартному submit

Подсказка: Сначала обработка события, затем отправка, затем получение и вывод.

Ответ:

- Установить слушатель события submit
- Препятствовать стандартному submit
- Отправить HTML-код и параметры на сервер
- Получить ответ от сервера
- Отобразить результаты пользователю

 **Шаг 13:**

Миссия 13

✂ Расставьте шаги, если пользователь меняет режим проверки (soft/strict):

1. Перезапустить проверку с новым режимом
2. Обработать событие onchange select
3. Обновить параметр запроса
4. Показать пользователю переключатель режима

Подсказка: Пользователь сначала видит выбор, затем onchange, потом обновление параметров и запуск проверки.

Ответ:

- Показать пользователю переключатель режима
- Обработать событие onchange select
- Обновить параметр запроса

- Перезапустить проверку с новым режимом

Шаг 14:

Миссия 14

 Расположите полную последовательность действий веб-интерфейса LocalAI от загрузки страницы до вывода результата:

1. Загрузить HTML-страницу с формой
2. Пользователь вводит код в textarea
3. Пользователь нажимает кнопку Проверить (submit)
4. preventDefault() отменяет перезагрузку
5. fetch отправляет запрос
6. Сервер отвечает JSON
7. Разобрать JSON-ответ
8. Подсветить ошибки
9. Отобразить результаты

Подсказка: Действия идут последовательно от инициализации к выводу.

Ответ:

- Загрузить HTML-страницу с формой
- Пользователь вводит код в textarea
- Пользователь нажимает кнопку Проверить (submit)
- preventDefault() отменяет перезагрузку
- fetch отправляет запрос
- Сервер отвечает JSON
- Разобрать JSON-ответ
- Подсветить ошибки
- Отобразить результаты

Модуль 7 - Модуль 7. Расширенные возможности

Урок 1 - Создание агентов для разных типов проверки

Урок 2 - Модульная структура (HTML QA Chain)

Урок 3 - Добавление памяти в LocalAI

Урок 4 - Пакетная обработка HTML-файлов

Урок 5 - Генерация отчётов (PDF, JSON, HTML)

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

Когда программист Мария запускает LocalAI для проверки HTML-кода, что она ожидает увидеть первым?

Подсказка: Обычно система быстро сообщает об общей корректности или наличии ошибок.

Ответ:

- **Сообщение о статусе анализа (успешно или есть ошибки)**
- Сразу подробный лог исправлений
- Интерфейс редактирования HTML

Шаг 02:

Миссия 2

Алексей получает сообщение от LocalAI: «Тег `<img>` не имеет атрибута `alt` ». Что ему следует сделать?

Подсказка: Атрибут `alt` важен для доступности.

Ответ:

- **Добавить подходящий атрибут `alt`**
- Удалить тег `<img>`
- Игнорировать предупреждение

Шаг 03:

Миссия 3

При проверке заметки блога система LocalAI сообщает: «Некорректный nesting: `<p>` внутри `<p>` ». Что самая разумная реакция?

Подсказка: HTML не допускает вложенные абзацы.

Ответ:

- **Разделить на два `<p>` или вложить подходящий контейнер**
- Закрыть внешний `<p>` раньше
- Оставить как есть

Шаг 04:

Миссия 4

В отчете LocalAI отмечено: «Незакрытый тег `<li>` ». Что это значит?

Подсказка: Каждый `<li>` должен иметь соответствующий закрывающий тег.

Ответ:

- **Нужно добавить закрывающий `</li>`**
- Удалить все `<li>`
- Пропустить и продолжить

Шаг 05:

Миссия 5

Сопоставьте сообщение LocalAI с наиболее вероятным действием разработчика.

Сообщения:

Ответ:

- 1. «Тег `<div>` закрыт раньше
- 2. «Атрибут `href` отсутствует»
- 3. «Нестандартный символ `&` в тексте без экранирования»
- 4. «Дублирование идентификатора `id`»

Действия:

1. Открыть и закрыть теги корректно
2. Добавить атрибут href с подходящей ссылкой
3. Заменить символ & на & или по контексту
4. Переименовать один из id, чтобы не дублировались

Подсказка:

Читайте сообщение LocalAI внимательно — оно указывает причину и как можно исправить.

- **1. «Тег <div> закрыт раньше** — Открыть и закрыть теги корректно
- **2. «Атрибут href отсутствует** — Добавить атрибут href с подходящей ссылкой
- **3. «Нестандартный символ & в тексте без экранирования** — Заменить символ & на & или по контексту
- **4. «Дублирование идентификатора id** — Переименовать один из id, чтобы не дублировались

🔮 Шаг 06:

Миссия 6

Сопоставьте тип ошибки HTML с примером, который LocalAI может обнаружить.

Тип ошибки:

Ответ:

- 1. Незакрытый тег
- 2. Неправильный nesting
- 3. Атрибут без значения
- 4. Дублирование id

Примеры:

1. `<ul><li>Item</ul>`
2. `<p><div>Text</p></div>`
3. `<input disabled>`
4. `<div id="a"><span id="a">`

Подсказка:

Каждое сообщение LocalAI обычно точно описывает, в чём ошибка — ищите соответствие по ключевым словам.

- **1. Незакрытый тег** — `<ul><li>Item</ul>`
- **2. Неправильный nesting** — `<p><div>Text</p></div>`
- **3. Атрибут без значения** — `<input disabled>`
- **4. Дублирование id** — `<div id="a"><span id="a">`

🔮 Шаг 07:

Миссия 7

Сопоставьте подсказку LocalAI с ключевым словом, которое она может искать.

Подсказки:

Ответ:

- 1. «unexpected closing tag»
- 2. «missing attribute value»
- 3. «duplicate id detected»

- 4. «invalid nesting detected»

Ключевые слова:

1. закрывающий тег не к месту
2. атрибут без значения
3. дублирование идентификаторов
4. неверное вложение тегов

Подсказка:

Ключевые слова часто буквально включены в подсказку от LocalAI.

- **1. «unexpected closing tag»** — закрывающий тег не к месту
- **2. «missing attribute value»** — атрибут без значения
- **3. «duplicate id detected»** — дублирование идентификаторов
- **4. «invalid nesting detected»** — неверное вложение тегов

 Шаг 08:

Миссия 8

Сопоставьте шаг из рабочего процесса LocalAI с его назначением.

Шаг:

Ответ:

- 1. Лексический анализ
- 2. Построение DOM-дерева
- 3. Семантическая проверка
- 4. Генерация отчёта об ошибках

Назначение:

1. Построить структуру тегов из токенов
2. Проверить вложение и правила HTML
3. Разбить исходный текст на токены
4. Выдать подробный отчет пользователю

Подсказка:

Подумайте, что должна делать система в каком порядке — сначала анализ, потом структурирование, затем проверки, и в конце вывод.

- **1. Лексический анализ** — Разбить исходный текст на токены
- **2. Построение DOM-дерева** — Построить структуру тегов из токенов
- **3. Семантическая проверка** — Проверить вложение и правила HTML
- **4. Генерация отчёта об ошибках** — Выдать подробный отчет пользователю

 Шаг 09:

Миссия 9

Расположите этапы работы LocalAI в правильном порядке:

1. Построение DOM-дерева
2. Генерация отчёта об ошибках
3. Лексический анализ
4. Семантическая проверка

Подсказка:

Логика: сначала текст → токены → дерево → проверки → отчёт.

Ответ:

- Построение DOM-дерева
- Генерация отчёта об ошибках
- Лексический анализ
- Семантическая проверка

Шаг 10:

Миссия 10

Упорядочите виды ошибок по степени сложности диагностики (от простого к сложному):

1. Незакрытый тег
2. Дублирование id
3. Неверное вложение тегов
4. Атрибут без значения

Подсказка:

Думайте, какие ошибки легко заметить синтаксически, а какие требуют контекста.

Ответ:

- Незакрытый тег
- Дублирование id
- Неверное вложение тегов
- Атрибут без значения

Шаг 11:

Миссия 11

Упорядочите шаги программиста при исправлении ошибки LocalAI:

1. Перепроверить исходный код
2. Запустить LocalAI ещё раз
3. Внести правки
4. Прочитать сообщение об ошибке

Подсказка:

Действия: увидеть сообщение, пересмотреть код, внести исправления, затем снова проверить.

Ответ:

- Перепроверить исходный код
- Запустить LocalAI ещё раз
- Внести правки
- Прочитать сообщение об ошибке

Шаг 12:

Миссия 12

Упорядочите уровни сложности проверки LocalAI (от базового к продвинутому):

1. Синтаксический анализ
2. Проверка атрибутов
3. Проверка вложения элементов

4. Проверка семантики и дублирования

Подсказка:

Система сначала проверяет базовый синтаксис, потом атрибуты, вложение, и в конце — семантику.

Ответ:

- Синтаксический анализ
- Проверка атрибутов
- Проверка вложения элементов
- Проверка семантики и дублирования

Шаг 13:

Миссия 13

Упорядочите сценарии взаимодействия программиста и LocalAI:

1. Программист получает отчёт об ошибках
2. Программист правит HTML-код
Программист запускает LocalAI
3. LocalAI анализирует код

Подсказка:

Последовательность естественная: отправка → анализ → результат → правка.

Ответ:

- Программист получает отчёт об ошибках
- Программист правит HTML-код
- Программист запускает LocalAI
- LocalAI анализирует код

Шаг 14:

Миссия 14

Расставьте фильтры, которые LocalAI применяет к сообщениям ошибок, в порядок от многих ложных срабатываний к строгим:

1. Сообщения уровня INFO
2. Предупреждения (WARNING)
3. Ошибки (ERROR)
4. Критические ошибки (CRITICAL)

Подсказка:

Сначала менее значимые, потом — важные.

Ответ:

- Сообщения уровня INFO
- Предупреждения (WARNING)
- Ошибки (ERROR)
- Критические ошибки (CRITICAL)

Шаг 15:

Миссия 15

Упорядочите виды проверок LocalAI по возрастанию «глубины» анализа:

1. Синтаксис (токены/теги)
2. Структура вложения элементов
3. Проверка атрибутов и значений
4. Проверка семантики и дублирования

Подсказка: Глубже — значит больше контекста и сложных зависимостей.

Ответ:

- Синтаксис (токены/теги)
- Структура вложения элементов
- Проверка атрибутов и значений
- Проверка семантики и дублирования

Шаг 16:

Миссия 16

Расположите стадии развития проекта LocalAI (от начальной до продвинутой):

1. Базовая проверка тегов
2. Проверка атрибутов и значений
3. Наложение семантических правил
4. Автоматическая коррекция и предложения

Подсказка: Проект развивается от простого к сложному: сначала синтаксис, потом атрибуты, затем семантика, и в конце — автоматические исправления.

Ответ:

- Базовая проверка тегов
- Проверка атрибутов и значений
- Наложение семантических правил
- Автоматическая коррекция и предложения

Шаг 17:

Миссия 17

Упорядочите ключевые этапы, когда нейросеть LocalAI исправляет HTML автоматически:

1. Идентификация ошибки
2. Предложение корректировки
3. Проверка, что новое решение валидно
4. Применение исправления к коду

Подсказка: Сначала ошибка → предложение → проверка → внедрение.

Ответ:

- Идентификация ошибки
- Предложение корректировки
- Проверка, что новое решение валидно
- Применение исправления к коду

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Мария получит предупреждение: «Attribute “class” without value». Какой из вариантов — корректный?

Подсказка: Атрибутам без значения часто ставят пустую строку или явное значение.

Ответ:

- `class=""`
- `class`
- `class=`

Шаг 02:

Миссия 2

При проверке блока меню LocalAI пишет: «Missing <nav> around navigation links». Что следует сделать?

Подсказка: Семантические теги помогают структуре документа.

Ответ:

- **Обернуть ссылки в тег <nav>**
- Удалить все ссылки
- Игнорировать предупреждение

Шаг 03:

Миссия 3

Сопоставьте тип предупреждения LocalAI с его значением:

Тип:

Ответ:

- 1. WARNING
- 2. ERROR
- 3. INFO

Значения:

1. Не критичное, полезное замечание
2. Ошибка, требующая исправления
3. Информационное сообщение без обязательных действий

Подсказка: Уровни сообщений часто стандартизированы.

- **1. WARNING** — Не критичное, полезное замечание
- **2. ERROR** — Ошибка, требующая исправления
- **3. INFO** — Информационное сообщение без обязательных действий

Шаг 04:

Миссия 4

Сопоставьте сообщение LocalAI с возможным местом ошибки в коде:

Сообщения:

Ответ:

- 1. «Unclosed tag <section> »
- 2. «Missing required attribute src on »
- 3. «Duplicate id "header" found»

Места кода:

1. В конце документа, не закрыли </section>
2. Тег без атрибута src
3. Два элемента с id="header"

Подсказка: Даже часть сообщения — полезный ориентир.

- **1. «Unclosed tag <section> »** — В конце документа, не закрыли </section>
- **2. «Missing required attribute src on »** — Тег без атрибута src
- **3. «Duplicate id "header" found»** — Два элемента с id="header"

 Шаг 05:

Миссия 5

Сопоставьте ключевой термин с его значением:

Термины:

Ответ:

- 1. Sanitization
- 2. Autocorrection
- 3. Heuristic suggestion

Значения:

1. Автоматическое исправление ошибок в коде
2. Предложение возможных вариантов на основе эвристики
3. Обработка входного текста для безопасности / корректности

Подсказка: Эти термины часто употребляют в системах анализа кода.

- **1. Sanitization** — Обработка входного текста для безопасности / корректности
- **2. Autocorrection** — Автоматическое исправление ошибок в коде
- **3. Heuristic suggestion** — Предложение возможных вариантов на основе эвристики

 Шаг 06:

Миссия 6

Сопоставьте стадию работы LocalAI с задачей:

Стадии:

Ответ:

- 1. Tokenization
- 2. Repair engine
- 3. Validation
- 4. Suggestion ranking

Задачи:

1. Создание списка токенов из HTML-кода
2. Проверка корректности предложенного исправления

3. Применение алгоритмов исправления ошибок
4. Выбор наилучшего варианта из списка

Подсказка: Стадии следуют от разбора → исправление → проверка → выбор.

- **1. Tokenization** — Создание списка токенов из HTML-кода
- **2. Repair engine** — Применение алгоритмов исправления ошибок
- **3. Validation** — Проверка корректности предложенного исправления
- **4. Suggestion ranking** — Выбор наилучшего варианта из списка

Шаг 07:

Миссия 7

Упорядочите приоритеты системы коррекции LocalAI:

1. Не ломать существующую структуру
2. Исправление синтаксиса
3. Сохранение семантики
4. Предложения альтернатив

Подсказка: Приоритеты обычно от обязательного к рекомендательному.

Ответ:

- Не ломать существующую структуру
- Исправление синтаксиса
- Сохранение семантики
- Предложения альтернатив

Шаг 08:

Миссия 8

Расположите этапы отладки подсистемы автокоррекции:

1. Сбор примеров ошибок
2. Тестирование на реальных страницах
3. Настройка алгоритмов
4. Оценка качества исправлений

Подсказка: Подумайте, как проходит цикл разработки и тестирования.

Ответ:

- Сбор примеров ошибок
- Тестирование на реальных страницах
- Настройка алгоритмов
- Оценка качества исправлений

Шаг 09:

Миссия 9

Упорядочите уровни глубины анализа от простейшего к самому сложному:

1. Синтаксический разбор
2. Анализ атрибутов
3. Контекстный анализ
4. Семантический анализ

Подсказка: Чем дальше — тем больше зависит от содержимого и смысла.

Ответ:

- Синтаксический разбор
- Анализ атрибутов
- Контекстный анализ
- Семантический анализ

Шаг 10:

Миссия 10

Расположите элементы, которые анализируются при автокоррекции:

1. Токены и теги
2. Имена атрибутов и значения
3. Вложенность и структура
4. Смысловое соответствие

Подсказка: Автокоррекция сначала смотрит синтаксис, потом структуру, а затем смысл.

Ответ:

- Токены и теги
- Имена атрибутов и значения
- Вложенность и структура
- Смысловое соответствие

Шаг 11:

Миссия 11

Упорядочите этапы системы обучения модели исправлений:

1. Сбор данных об ошибках
2. Размечивание правильных исправлений
3. Обучение модели на примерах
4. Оценка на тестовом наборе

Подсказка: Подумайте как в ML-проекте: от данных к обучению к проверке.

Ответ:

- Сбор данных об ошибках
- Размечивание правильных исправлений
- Обучение модели на примерах
- Оценка на тестовом наборе

Шаг 12:

Миссия 12

Расположите фазы взаимодействия пользователя с подсказками автокоррекции:

1. Показ предложений исправлений
2. Пользователь выбирает вариант
3. LocalAI проверяет выбранный вариант
4. Применение выбранного исправления

Подсказка: Пользователь участвует между предложением и применением.

Ответ:

- Показ предложений исправлений
- Пользователь выбирает вариант
- LocalAI проверяет выбранный вариант
- Применение выбранного исправления

Шаг 13:

Миссия 13

Упорядочите элементы отчёта LocalAI по степени важности:

1. Критические ошибки
2. Ошибки
3. Предупреждения
4. Информационные сообщения

Подсказка: В отчётах сначала показывают самые серьёзные проблемы.

Ответ:

- Критические ошибки
- Ошибки
- Предупреждения
- Информационные сообщения

Шаг 14:

Миссия 14

Расположите шаги, когда LocalAI отвечает на запрос пользователя:

1. Пользователь загружает HTML
2. LocalAI анализирует
3. LocalAI предлагает исправления
4. Пользователь принимает или отклоняет исправления

Подсказка: Это цикл: загрузка → анализ → предложения → принятие.

Ответ:

- Пользователь загружает HTML
- LocalAI анализирует
- LocalAI предлагает исправления
- Пользователь принимает или отклоняет исправления

Шаг 15:

Миссия 15

Мария получит предупреждение: «Attribute “class” without value». Какой из вариантов — корректный?

Подсказка: Атрибутам без значения часто ставят пустую строку или явное значение.

Ответ:

- `class=""`
- `class`
- `class=`

🔮 Шаг 16:

Миссия 16

При проверке блока меню LocalAI пишет: «Missing <nav> around navigation links». Что следует сделать?

Подсказка: Семантические теги помогают структуре документа.

Ответ:

- **Обернуть ссылки в тег <nav>**
- Удалить все ссылки
- Игнорировать предупреждение

🔮 Шаг 17:

Миссия 17

Сопоставьте тип предупреждения LocalAI с его значением:

Тип:

Ответ:

- 1. WARNING
- 2. ERROR
- 3. INFO

Значения:

1. Не критичное, полезное замечание
2. Ошибка, требующая исправления
3. Информационное сообщение без обязательных действий

Подсказка: Уровни сообщений часто стандартизированы.

- **1. WARNING** — Не критичное, полезное замечание
- **2. ERROR** — Ошибка, требующая исправления
- **3. INFO** — Информационное сообщение без обязательных действий

🔮 Шаг 18:

Миссия 18

Сопоставьте сообщение LocalAI с возможным местом ошибки в коде:

Сообщения:

Ответ:

- 1. «Unclosed tag <section> »
- 2. «Missing required attribute src on »
- 3. «Duplicate id "header" found»

Места кода:

1. В конце документа, не закрыли </section>
2. Тег без атрибута src
3. Два элемента с id="header"

Подсказка: Даже часть сообщения — полезный ориентир.

- **1. «Unclosed tag <section> »** — В конце документа, не закрыли </section>
- **2. «Missing required attribute src on »** — Тег без атрибута src

- **3. «Duplicate id “header” found»** — Два элемента с id="header"

Шаг 19:

Миссия 19

Сопоставьте ключевой термин с его значением:

Термины:

Ответ:

- 1. Sanitization
- 2. Autocorrection
- 3. Heuristic suggestion

Значения:

1. Автоматическое исправление ошибок в коде
2. Предложение возможных вариантов на основе эвристики
3. Обработка входного текста для безопасности / корректности

Подсказка: Эти термины часто употребляют в системах анализа кода.

- **1. Sanitization** — Обработка входного текста для безопасности / корректности
- **2. Autocorrection** — Автоматическое исправление ошибок в коде
- **3. Heuristic suggestion** — Предложение возможных вариантов на основе эвристики

Модуль 8 - Модуль 8. Оптимизация и автоматизация

Урок 1 - Оптимизация модели (quantization Q4Q5)

Урок 2 - Тонкая настройка и дообучение (fine-tuning)

Урок 3 - Кеширование ответов

Урок 4 - Автоматические проверки через cron

Урок 5 - Интеграция в GitHub Actions или GitLab CI

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

 Оля хочет уменьшить размер HTML-файла: что первое стоит минифицировать?

Подсказка: пробелы, лишние переносы — лёгкая цель.

Ответ:

- **Удалить лишние пробелы и переносы строк**
- Удалить весь визуальный контент
- Убрать DOCTYPE
- Убрать полностью

Шаг 02:

Миссия 2

 Дмитрий хочет, чтобы его HTML корректно проверял LocalAI: что нужно сделать?

Подсказка: валидность — ключ к автоматической проверке.

Ответ:

- **Проверить HTML на валидность перед загрузкой в LocalAI**
- **Отправлять невалидный HTML** — пусть исправляет сам
- **Не проверять ничего** — больше эксперимент

💡 Шаг 03:

Миссия 3

🔍 Сергей редактирует HTML и хочет сопоставить следующие термины с их описаниями:

Термины:

Ответ:

- Lazy loading
- Viewport meta
- Minification
- Semantic HTML

Определения:

1. Минимизация пробелов, комментариев, переносов
2. Отложенная загрузка изображений по мере прокрутки
3. Использование правильных семантических тегов (например,)
4. Метатег для управления масштабом на мобильных устройствах

Подсказка: вспомните, как быстро загружаются страницы на телефоне.

- **Lazy loading** — Отложенная загрузка изображений по мере прокрутки
- **Viewport meta** — Метатег для управления масштабом на мобильных устройствах
- **Minification** — Минимизация пробелов, комментариев, переносов
- **Semantic HTML** — Использование правильных семантических тегов (например,)

💡 Шаг 04:

Миссия 4

🔍 Ксения хочет сопоставить ошибки HTML и способы их автоматического исправления LocalAI:

Ошибки:

Ответ:

- Непарный тег
- Неправильный атрибут
- Отсутствие alt у img
- Дублирование id

Способы исправления:

1. Добавить закрывающий тег
2. Удалить или заменить атрибут
3. Добавить атрибут alt с описанием
4. Уникализировать идентификаторы элементов

Подсказка: подумайте, как LocalAI может автоматически исправлять распространённые ошибки.

- **Непарный тег** — Добавить закрывающий тег

- **Неправильный атрибут** — Удалить или заменить атрибут
- **Отсутствие alt у img** — Добавить атрибут alt с описанием
- **Дублирование id** — Уникализировать идентификаторы элементов

🔮 Шаг 05:

Миссия 5

♥ Николай хочет понять, какие проверки выполняет LocalAI — сопоставьте:

Проверки:

Ответ:

- Валидность тегов
- Наличие заголовков h1-h6
- Обнаружение пустых контейнеров
- Обнаружение дубликатов классов

Действия LocalAI:

1. Предложить удалить пустые блоки
2. Проверить корректность структуры тегов
3. Указать конфликты в классах элементов
4. Проверить наличие и иерархию заголовков

Подсказка: LocalAI анализирует структуру и семантику.

- **Валидность тегов** — Проверить корректность структуры тегов
- **Наличие заголовков h1-h6** — Проверить наличие и иерархию заголовков
- **Обнаружение пустых контейнеров** — Предложить удалить пустые блоки
- **Обнаружение дубликатов классов** — Указать конфликты в классах элементов

🔮 Шаг 06:

Миссия 6

Расположите этапы обработки HTML LocalAI в правильном порядке:

1. Исправление синтаксических ошибок
2. Проверка семантики и заголовков
3. Удаление пустых контейнеров и избыточного кода
4. Отчёт об изменениях пользователю

Подсказка: сначала устраним синтаксис, потом — смысл, после — чистка, и лишь затем — отчёт.

Ответ:

- Исправление синтаксических ошибок
- Проверка семантики и заголовков
- Удаление пустых контейнеров и избыточного кода
- Отчёт об изменениях пользователю

🔮 Шаг 07:

Миссия 7

⚙ Представим, что HTML нужно автоматом оптимизировать: расположите по порядку шаги автоматизации:

1. Сканирование HTML-файла
2. Применение шаблонов исправлений
3. Запрос пользователя о внесении изменений
4. Сохранение оптимизированной версии

Подсказка: сначала анализ, затем шаблоны, потом подтверждение, потом сохранение.

Ответ:

- Сканирование HTML-файла
- Применение шаблонов исправлений
- Запрос пользователя о внесении изменений
- Сохранение оптимизированной версии

🔗 Шаг 08:

Миссия 8

🌀 Представьте цепочку от получения HTML до финальной версии LocalAI:

1. Пользователь присылает HTML
2. LocalAI анализирует ошибки
3. LocalAI предлагает исправления
4. Пользователь утверждает изменения
5. LocalAI выдает финальную исправленную HTML

Подсказка: последовательность: вход → анализ → предложение → одобрение → выход.

Ответ:

- Пользователь присылает HTML
- LocalAI анализирует ошибки
- LocalAI предлагает исправления
- Пользователь утверждает изменения
- LocalAI выдает финальную исправленную HTML

🔗 Шаг 09:

Миссия 9

🔗 Что происходит внутри LocalAI: упорядочите модули по порядку обработки:

1. Лексический разбор
2. Синтаксический анализ
3. Семантическая проверка
4. Генерация корректур
5. Вывод пользователю

Подсказка: сначала лексика, потом синтаксис, затем смысл, затем исправления и вывод.

Ответ:

- Лексический разбор
- Синтаксический анализ
- Семантическая проверка
- Генерация корректур
- Вывод пользователю

Шаг 10:

Миссия 10

 В цепочке CI/CD: как встроить проверку LocalAI? Упорядочите шаги:

1. Закоммитить HTML-код
2. Запустить LocalAI-проверку в pipeline
3. Внести правки по отчёту
4. Запустить тесты и сборку
5. Выкатить на прод

Подсказка: сначала код, затем проверка, затем правки, тесты и публикация.

Ответ:

- Закоммитить HTML-код
- Запустить LocalAI-проверку в pipeline
- Внести правки по отчёту
- Запустить тесты и сборку
- Выкатить на прод

Шаг 11:

Миссия 11

Разработчики часто встраивают LocalAI в редактор. Расположите стадии автокоррекции:

1. Пользователь печатает HTML
2. Редактор отправляет фрагмент в LocalAI
3. LocalAI анализирует фрагмент
4. Предлагает подсказку и исправления в редакторе
5. Пользователь принимает или отклоняет

Подсказка: поток: ввод → отправка → анализ → предложение → решение.

Ответ:

- Пользователь печатает HTML
- Редактор отправляет фрагмент в LocalAI
- LocalAI анализирует фрагмент
- Предлагает подсказку и исправления в редакторе
- Пользователь принимает или отклоняет

Шаг 12:

Миссия 12

 При большом объёме HTML: как LocalAI оптимизирует? Упорядочите этапы:

1. Разбиение на чанки
2. Параллельная проверка чанков
3. Слияние отчётов в один
4. Предложение исправлений пользователю
5. Применение одобренных правок

Подсказка: большие файлы требуют деления и объединения.

Ответ:

- Разбиение на чанки

- Параллельная проверка чанков
- Слияние отчётов в один
- Предложение исправлений пользователю
- Применение одобренных правок

Шаг 13:

Миссия 13

 При автоматическом исправлении HTML: расположите приоритеты LocalAI:

1. Синтаксис ошибок (не закрытые теги)
2. Ошибки вложенности (невалидная структура)
3. Семантика и заголовки
4. Избыточность классов и дублирования

Подсказка: сначала устранить базовые ошибки, затем — смысл и детали.

Ответ:

- Синтаксис ошибок (не закрытые теги)
- Ошибки вложенности (невалидная структура)
- Семантика и заголовки
- Избыточность классов и дублирования

Шаг 14:

Миссия 14

 Финальное испытание: упорядочите полный pipeline LocalAI-автоматизации:

1. Получение HTML от пользователя
2. Парсинг и лексический анализ
3. Применение правил исправлений
4. Проверка семантики и структуры
5. Генерация отчёта и предложений
6. Утверждение пользователем
7. Сохранение исправленного HTML

Подсказка: начинайте с приёма и анализа, далее правила, проверка, предложение, согласование и финал.

Ответ:

- Получение HTML от пользователя
- Парсинг и лексический анализ
- Применение правил исправлений
- Проверка семантики и структуры
- Генерация отчёта и предложений
- Утверждение пользователем
- Сохранение исправленного HTML

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

Инженер Анна использует LocalAI, чтобы проверить HTML-файл на ошибки: какая из проверок будет ****первой****?

Подсказка: Начать стоит с базового — синтаксического уровня.

Ответ:

- **Проверка на незакрытые теги**
- Проверка оптимизации CSS
- Проверка логики JavaScript

🔮 Шаг 02:

Миссия 2

При проверке шаблона LocalAI выдал сообщение: «атрибут `align`` устарел». Что лучше сделать?

Подсказка: HTML5 рекомендует использовать CSS, а не устаревшие атрибуты.

Ответ:

- **Убрать атрибут `align`` и применить CSS-стили**
- **Оставить атрибут** — это не критично
- Замаскировать атрибут через JavaScript

🔮 Шаг 03:

Миссия 3

Программистка Марина хочет, чтобы LocalAI автоматически исправлял мелкие опечатки в тегах (``` → ```). Какой режим выбрать?

Подсказка: Ищите режим «автоисправление» или «autofix».

Ответ:

- **Режим автокоррекции ошибок**
- Только отчет без исправлений
- Режим «только предупреждения»

🔮 Шаг 04:

Миссия 4

👁 Лена видит в отчёте LocalAI: «лишний закрывающий тег ```». Сопоставьте тип ошибки и описания:

Типы:

Ответ:

- Избыточный элемент
- Недостающий элемент
- Неправильный атрибут
- Слишком глубокая вложенность

Описания:

1. Тег закрыт, но не был открыт
2. Тег открыт, но не закрыт
3. Атрибут не соответствует стандарту
4. Наслоение многих блоков без смысла

Подсказка: Лишний `` — это закрытие без открытия.

- **Избыточный элемент** — Тег закрыт, но не был открыт
- **Недостающий элемент** — Тег открыт, но не закрыт
- **Неправильный атрибут** — Атрибут не соответствует стандарту
- **Слишком глубокая вложенность** — Наслоение многих блоков без смысла

🔧 Шаг 05:

Миссия 5

👤 Команда составила список атрибутов: сопоставьте атрибут и его статус в HTML5.

Атрибуты:

Ответ:

- `bgcolor`
- `onclick`
- `href`
- `novalidate`

Статусы:

1. Устаревший, не рекомендуется
2. Допускается, но с осторожностью
3. Обязательный для ссылок
4. Новый атрибут форм

Подсказка: `bgcolor` — из старого HTML, `novalidate` — из HTML5.

- **bgcolor** — Устаревший, не рекомендуется
- **onclick** — Допускается, но с осторожностью
- **href** — Обязательный для ссылок
- **novalidate** — Новый атрибут форм

🔧 Шаг 06:

Миссия 6

✂ Игорь хочет автоматизировать процесс: упорядочите этапы интеграции LocalAI в CI/CD.

1. Настроить правило падения сборки при критических ошибках
2. Добавить вызов LocalAI перед деплоем
3. Проанализировать отчет и уведомить команду
4. Обновить конфигурацию проверки (настройка порогов)

Подсказка: Сначала конфигурация, потом добавление в pipeline, потом выполнение и реакция.

Ответ:

- Обновить конфигурацию проверки (настройка порогов)
- Добавить вызов LocalAI перед деплоем
- Проанализировать отчет и уведомить команду
- Настроить правило падения сборки при критических ошибках

🔧 Шаг 07:

Миссия 7

☞ Настройка автоматического исправления: расставьте шаги в логическом порядке.

1. Запустить LocalAI в режиме autofix
2. Сохранить изменённый HTML
3. Сделать резервную копию исходного файла
4. Проверить, что визуальное отображение не изменилось

Подсказка: Делайте резервную копию перед изменениями.

Ответ:

- Сделать резервную копию исходного файла
- Запустить LocalAI в режиме autofix
- Сохранить изменённый HTML
- Проверить, что визуальное отображение не изменилось

🔮 Шаг 08:

Миссия 8

🔗 Процесс ревизии шаблонов: расставьте шаги от анализа до исправления.

1. Получить отчёт LocalAI
2. Выбрать наиболее критичные ошибки
3. Автоматически исправить тривиальные ошибки
4. Проверить результат вручную

Подсказка: Тривиальные ошибки решаются автоматически, остальные — вручную.

Ответ:

- Получить отчёт LocalAI
- Выбрать наиболее критичные ошибки
- Автоматически исправить тривиальные ошибки
- Проверить результат вручную

🔮 Шаг 09:

Миссия 9

🔗 В процессе CI: упорядочите шаги при откате изменений из LocalAI.

1. Отменить изменения, если тесты упали
2. Сохранить лог LocalAI и отчёт ошибок
3. Оповестить разработчика об ошибках
4. Возобновить сборку без автокоррекции

Подсказка: Никогда не удаляйте отчёт — он нужен для диагностики.

Ответ:

- Сохранить лог LocalAI и отчёт ошибок
- Оповестить разработчика об ошибках
- Отменить изменения, если тесты упали
- Возобновить сборку без автокоррекции

🔮 Шаг 10:

Миссия 10

🔧 Как отлаживать конфигурацию LocalAI: расставьте шаги.

1. Изменить порог чувствительности
2. Прогнать тестовый шаблон
3. Собрать отчет об ошибках
4. Вернуться к прежним настройкам, если неудачно

Подсказка: Начинайте с безопасных изменений порога.

Ответ:

- Изменить порог чувствительности
- Прогнать тестовый шаблон
- Собрать отчет об ошибках
- Вернуться к прежним настройкам, если неудачно

🔧 Шаг 11:

Миссия 11

📦 Команда хочет встроить LocalAI в git-хуки: упорядочите действия.

1. Написать скрипт запуска LocalAI при коммите
2. Добавить скрипт в husky или pre-commit hook
3. Протестировать на фиктивном изменении
4. Разрешить коммит, если проверка пройдена

Подсказка: Локальный запуск перед коммитом — ключ.

Ответ:

- Написать скрипт запуска LocalAI при коммите
- Добавить скрипт в husky или pre-commit hook
- Протестировать на фиктивном изменении
- Разрешить коммит, если проверка пройдена

🔧 Шаг 12:

Миссия 12

⚙️ Оптимизация отчётов: упорядочите этапы фильтрации ошибок перед выдачей пользователю.

1. Отключить непрактичные предупреждения
2. Группировать одинаковые ошибки
3. Сортировать по критичности
4. Форматировать вывод с контекстом

Подсказка: Удобство — в группировке и сортировке перед выводом.

Ответ:

- Отключить непрактичные предупреждения
- Группировать одинаковые ошибки
- Сортировать по критичности
- Форматировать вывод с контекстом

🔧 Шаг 13:

Миссия 13

🔗 Полный сценарий встраивания LocalAI: расставьте этапы от начала до конца.

1. Установить LocalAI как зависимость
2. Сконфигурировать правила проверки
3. Добавить этап проверки в CI/CD
4. Настроить автокоррекцию мелких ошибок
5. Внедрить логику отката при сбое

Подсказка: Сначала установка и настройка, затем интеграция и логика отказа.

Ответ:

- Установить LocalAI как зависимость
- Сконфигурировать правила проверки
- Добавить этап проверки в CI/CD
- Настроить автокоррекцию мелких ошибок
- Внедрить логику отката при сбое

💡 **Шаг 14:**

Миссия 14

Коллега предлагает отключить проверки LocalAI, чтобы ускорить сборку. Что скажете?

Подсказка: Проверки важны для качества, но можно настроить порог строгости.

Ответ:

- **Лучше снизить уровень строгости, а не отключать проверку**
- Согласиться и убрать проверки
- Отключить всё, если сборка «фаст» важнее

📁 **Модуль 9 - Модуль 9. Проект AI HTML Auditor**

📖 **Урок 1 - Постановка задачи и структура проекта**

📖 **Урок 2 - Интеграция W3C и LocalAI в один поток**

📖 **Урок 3 - Исправление и форматирование HTML**

📖 **Урок 4 - Сохранение отчётов и результатов**

📖 **Урок 5 - Тестирование и финальная сборка проекта**

📖 **Урок 6 - Викторина 1**

💡 **Шаг 01:**

Миссия 1

🖥️ Инга, фронтендер, загрузила HTML-страницу с неправильным тегом: она закрыла `<div>` как `</section>`. Какая реакция нейросети в LocalAI будет наиболее уместной?

Подсказка: Нейросеть анализирует соответствие открывающих и закрывающих тегов.

Ответ:

- **Предложить заменить `</section>` на `</div>`**
- Удалить весь `<div>`-блок
- **Оставить как есть** — это не критично

🔮 Шаг 02:

Миссия 2

✦ Алиса вставила атрибут `align="center"` в `<img>`. Что предложит LocalAI?

Подсказка: Используйте CSS вместо устаревших атрибутов.

Ответ:

- Предложить убрать `align` и применить CSS свойство `text-align` или `margin`
- Оставить `align` как резерв
- Заменить на `valign`

🔮 Шаг 03:

Миссия 3

🌐 Борис добавил атрибут `lang="en"` в тег `<html>`. Это корректно, но что ещё могла бы подсказать нейросеть?

Подсказка: Язык страницы важен для SEO и доступности.

Ответ:

- Проверить, совпадает ли язык с содержанием текста
- Убрать атрибут `lang`
- Изменить на `lang="ru"` без проверки

🔮 Шаг 04:

Миссия 4

🔍 Сопоставьте HTML-проблему и её симптом, который мог бы обнаружить LocalAI.

Проблемы:

Ответ:

- Непарные теги
- Устаревший атрибут
- Отсутствие `alt` в `<img>`
- Неправильное вложение тегов

Симптомы:

1. Изображение без описания для скринридера
2. Тег закрывается не там, где открыт
3. Использование `align`, `bgcolor` и т.п.
4. Блочный элемент внутри строчного без обертки

Подсказка: Подумайте, какой симптом соответствует каждой из типичных ошибок.

- **Непарные теги** — Тег закрывается не там, где открыт
- **Устаревший атрибут** — Использование `align`, `bgcolor` и т.п.
- **Отсутствие `alt` в** — Изображение без описания для скринридера
- **Неправильное вложение тегов** — Блочный элемент внутри строчного без обертки

🔮 Шаг 05:

Миссия 5

✦ Разработчик Иван использует LocalAI для анализа форм на странице. Сопоставьте проблему и её проявление.

Проблемы:

Ответ:

- Отсутствие атрибута `name` в `input`
- Несколько одинаковых `id`
- Использование `<table>` вместо `div`-сеток
- Форма внутри ссылки

Проявления:

1. Невозможность передать значение с сервера по имени поля
2. CSS/JS ломается, потому что `id` не уникальны
3. Верстка выглядит как таблица, плохо адаптируется
4. Клик по форме ведёт на страницу ссылки

Подсказка: Подумайте о том, что делает каждый атрибут, тег или вложенность.

- **Отсутствие атрибута `name` в `input`** — Невозможность передать значение с сервера по имени поля
- **Несколько одинаковых `id`** — CSS/JS ломается, потому что `id` не уникальны
- **Использование вместо `div`-сеток** — Верстка выглядит как таблица, плохо адаптируется
- **Форма внутри ссылки** — Клик по форме ведёт на страницу ссылки

Шаг 06:

Миссия 6

🔗 У Максима есть последовательность правок, которые хочет предложить LocalAI для HTML страницы. Расположите их в логическом порядке:

1. Проверка `alt` у изображений
2. Валидация вложений тегов
3. Удаление устаревших атрибутов
4. Проверка парности тегов

Подсказка: Начинать стоит с базовой структуры, потом дополнять детали.

Ответ:

- Проверка парности тегов
- Валидация вложений тегов
- Удаление устаревших атрибутов
- Проверка `alt` у изображений

Шаг 07:

Миссия 7

Где должна быть проверка `accessibility` (доступности) в цепочке аудита? Упорядочите этапы:

1. Синтаксическая проверка HTML
2. Проверка семантики тегов
3. Проверка атрибутов (`alt`, `title` и т.д.)
4. Проверка соответствия ARIA и доступности

Подсказка: Доступность — это финальный штрих к качественному HTML.

Ответ:

- Синтаксическая проверка HTML
- Проверка семантики тегов
- Проверка атрибутов (alt, title и т.д.)
- Проверка соответствия ARIA и доступности

 Шаг 08:

Миссия 8

 Расположите уровни анализа LocalAI при проверке проекта (от общего к частному):

1. Контент и семантика
2. Структура тегов
3. Синтаксис HTML
4. Атрибуты и метаданные

Подсказка: Сначала базовая валидность, потом детали.

Ответ:

- Синтаксис HTML
- Структура тегов
- Атрибуты и метаданные
- Контент и семантика

 Шаг 09:

Миссия 9

 При запуске полного аудита LocalAI делает этапы модификации. Расположите их в порядке:

1. Сгенерировать предложения по исправлению
2. Сохранить оригинал кода
3. Применить лучшие правки
4. Сравнить результат с исходником

Подсказка: Не забудьте резервную копию перед изменениями.

Ответ:

- Сохранить оригинал кода
- Сгенерировать предложения по исправлению
- Применить лучшие правки
- Сравнить результат с исходником

 Шаг 10:

Миссия 10

 Перед тем как править, LocalAI видит конфликт: две версии исправления для одной проблемы. Какую стратегию выбрать?

Подсказка: Важна правильность, а не просто агрессивное исправление.

Ответ:

- Предложить пользователю выбор между вариантами
- Автоматически применить первый вариант
- Отклонить оба варианта

🔗 Шаг 11:

Миссия 11

При проверке большого проекта из множества HTML-файлов LocalAI обнаружил несоответствия между шаблоном и страницами. Что предложить?

Подсказка: Важно сохранить единый стиль файлов.

Ответ:

- **Синхронизировать исправления шаблона со всеми страницами**
- Оставить страницы разными
- Игнорировать несоответствия

🔗 Шаг 12:

Миссия 12

🔗 Сопоставьте сложные ошибки, которые может уловить LocalAI, и подходы к их исправлению.

Ошибки:

Ответ:

- Смешение html и inline CSS
- Динамически генерируемые теги без проверки
- Непоследовательное использование классов
- Отсутствие fallback для медиа

Подходы:

1. Вынос стилей в CSS-файл
2. Проверка шаблонизатора генерации тегов
3. Настроить единые классы через CSS-конвенции
4. Добавить атрибуты srcset или альтернативное содержимое

Подсказка: Каждая ошибка диктует свою стратегию — не пытайтесь «универсальное» решение.

- **Смешение html и inline CSS** — Вынос стилей в CSS-файл
- **Динамически генерируемые теги без проверки** — Проверка шаблонизатора генерации тегов
- **Непоследовательное использование классов** — Настроить единые классы через CSS-конвенции
- **Отсутствие fallback для медиа** — Добавить атрибуты srcset или альтернативное содержимое

🔗 Шаг 13:

Миссия 13

🔗 Представьте, что проект состоит из трёх вкладок: index.html, about.html, contact.html. LocalAI должен выполнить аудит и внести исправления. Упорядочите шаги:

1. Проверить все HTML-файлы на синтаксис
2. Сгенерировать рекомендации по каждому файлу
3. Сопоставить изменения с шаблоном сайта
4. Применить и записать изменения

Подсказка: Сначала анализируем, потом генерируем, потом применяем.

Ответ:

- Проверить все HTML-файлы на синтаксис
- Сгенерировать рекомендации по каждому файлу
- Сопоставить изменения с шаблоном сайта
- Применить и записать изменения

 Шаг 14:

Миссия 14

 **Финальная задача:** LocalAI запускает цепочку исправлений для всего проекта. Расположите этапы в порядке:

1. Запуск синтаксического анализа
2. Семантический аудит тегов и контента
3. Проверка атрибутов и accessibility
4. Формирование окончательных патчей и применение
5. Сравнение итогового HTML с исходным

Подсказка: Представьте полный цикл аудита: от проверки до применения и контроля.

Ответ:

- Запуск синтаксического анализа
- Семантический аудит тегов и контента
- Проверка атрибутов и accessibility
- Формирование окончательных патчей и применение
- Сравнение итогового HTML с исходным

 Урок 7 - Викторина 2

 Шаг 01:

Миссия 1

 Сергей загрузил страницу, и LocalAI показывает: «неизвестный тег `<foo>`». Что он должен предложить?

Подсказка: Иногда опечатка — частая причина неизвестных тегов.

Ответ:

- Проверить, не опечатка ли `<foo>` и предложить `<div>` или `<span>`
- Признать `<foo>` допустимым
- Автоматически удалить тег `<foo>`

 Шаг 02:

Миссия 2

 Ирина вставила тег `<iframe>` без атрибута ``title``. Как LocalAI должен реагировать?

Подсказка: Для доступности `iframe` важно дать описание.

Ответ:

- Предложить добавить атрибут ``title`` с описанием содержимого
- Удалить `iframe`
- Оставить без изменений

🔮 Шаг 03:

Миссия 3

🔗 Павел сделал ссылку `<a href="#">` без текста. Что предлагает LocalAI?

Подсказка: Ссылки без текста бессмысленны для пользователей.

Ответ:

- Попросить добавить читаемый текст или `aria-label`
- Удалить ссылку
- Оставить как есть

🔮 Шаг 04:

Миссия 4

📦 Люба использует атрибут `style="display:none"` для скрытия контента. Что должен подсказать LocalAI?

Подсказка: Не злоупотребляйте встроенным стилем, особенно если это влияет на доступность.

Ответ:

- Предложить использовать CSS-класс с визуальным и доступным скрытием
- Оставить `inline style`
- Удалить элемент полностью

🔮 Шаг 05:

Миссия 5

📁 Алексей вложил блок `<div>` прямо в `<p>`. Как должен отреагировать LocalAI?

Подсказка: Блочные элементы не допускаются внутри параграфов.

Ответ:

- Предложить закрыть `<p>` перед `<div>`
- Разрешить вложение
- Удалить `<div>`

🔮 Шаг 06:

Миссия 6

Сопоставьте HTML-атрибут и его рекомендацию от LocalAI.

Атрибуты:

Ответ:

- `onclick`
- `width` и `height` в `<img>`
- `id` без уникальности
- `role` без контекста

Рекомендации:

1. Использовать CSS/JS-события через `addEventListener`
2. Задавать размеры через CSS или атрибуты для устойчивой верстки
3. Гарантировать уникальность `id` по странице
4. Указывать `role` только с подходящим семантическим контекстом

Подсказка: Подумайте, какой атрибут когда используется корректно.

- **onclick** — Использовать CSS/JS-события через addEventListener
- **width и height** — Задавать размеры через CSS или атрибуты для устойчивой верстки
- **id без уникальности** — Гарантировать уникальность id по странице
- **role без контекста** — Указывать role только с подходящим семантическим контекстом

🔗 Шаг 07:

Миссия 7

🔍 Иван создал форму, где <label> не ассоциирован с input. Сопоставьте ошибку и её последствие.

Ошибки:

Ответ:

- label без атрибута for
- input без id
- label вложен неправильно
- несоответствие for и id

Последствия:

1. Нельзя кликнуть на подпись, чтобы активировать поле
2. Нельзя найти соответствие между label и input
3. Влияние на доступность
4. Ошибочная ассоциация поля и текста

Подсказка: Label и input должны быть связаны корректно для UX и accessibility.

- **label без атрибута for** — Нельзя кликнуть на подпись, чтобы активировать поле
- **input без id** — Нельзя найти соответствие между label и input
- **label вложен неправильно** — Влияние на доступность
- **несоответствие for и id** — Ошибочная ассоциация поля и текста

🔗 Шаг 08:

Миссия 8

🌐 Сопоставьте семантический тег и его рекомендуемое использование.

Теги:

Ответ:

- <article>
- <aside>
- <header>
- <footer>

Использование:

1. Контент, независимый сам по себе
2. Боковая панель или доп. контент
3. Начало тематической секции (заголовок, навигация)
4. Конец или нижняя часть секции

Подсказка: Семантика — ключ к структурированному HTML.

- — Контент, независимый сам по себе

- — Боковая панель или доп. контент
- — Начало тематической секции (заголовок, навигация)
- — Конец или нижняя часть секции

Шаг 09:

Миссия 9

🔪 Дмитрий планирует шаги аудита LocalAI для одной страницы. Упорядочите этапы:

1. Проверка валидности тегов
2. Семантический аудит (section, article и др.)
3. Проверка атрибутов и доступности
4. Генерация патчей и применение

Подсказка: Сперва валидность, затем семантика, потом атрибуты и усовершенствования.

Ответ:

- Проверка валидности тегов
- Семантический аудит (section, article и др.)
- Проверка атрибутов и доступности
- Генерация патчей и применение

Шаг 10:

Миссия 10

🔪 В проекте множество страниц и шаблонов. Как LocalAI выполняет глобальный аудит? Упорядочите шаги:

1. Собрать карту всех HTML-файлов
2. Проверить каждый файл на синтаксис
3. Сопоставить шаблоны и общие элементы
4. Применить массовые правки

Подсказка: Начинаем с разбора структуры проекта, затем – проверка, потом применение.

Ответ:

- Собрать карта всех HTML-файлов
- Проверить каждый файл на синтаксис
- Сопоставить шаблоны и общие элементы
- Применить массовые правки

Шаг 11:

Миссия 11

⚙ LocalAI готовит комплексный патч для проекта. Расположите этапы:

1. Выявить конфликты между файлами
2. Сгенерировать предложения правок
3. Отфильтровать неподходящие правки
4. Применить окончательные патчи
5. Проверить итоговое состояние файлов

Подсказка: После генерации идёт фильтрация, затем применение и проверка.

Ответ:

- Выявить конфликты между файлами
- Сгенерировать предложения правок
- Отфильтровать неподходящие правки
- Применить окончательные патчи
- Проверить итоговое состояние файлов

 Шаг 12:

Миссия 12

🔗 При циклических правках LocalAI выполняет итерации. Упорядочите шаги одной итерации:

1. Анализ ошибок
2. Генерация правок
3. Применение ревизий
4. Оценка улучшения

Подсказка: Важно оценивать, что исправления действительно улучшают код.

Ответ:

- Анализ ошибок
- Генерация правок
- Применение ревизий
- Оценка улучшения

 Шаг 13:

Миссия 13

⚠ LocalAI обнаружил тег `<meta http-equiv>`, устаревший. Что делать?

Подсказка: Современные альтернативы иногда предпочтительнее устаревших конструкций.

Ответ:

- **Предложить заменить на современный тег или HTTP заголовок**
- Игнорировать предупреждение
- Удалить мета без замены

 Шаг 14:

Миссия 14

✨ Вера пишет компоненты, и LocalAI предлагает стандартизировать классы. Что — правильный подход?

Подсказка: Консистентность в именах классов облегчает поддержку.

Ответ:

- **Предложить соглашение BEM или иную систему именования**
- Оставить текущие разнообразные классы
- Удалить классы вовсе

 Шаг 15:

Миссия 15

🔗 Сопоставьте глубокую ошибку и стратегию исправления от LocalAI.

Ошибка:

Ответ:

- Вложенные формы
- Несоответствие DOCTYPE
- Смешение HTTP/HTTPS в ресурсах
- Отсутствие кеширования статических ресурсов

Стратегии:

1. Разделить формы, оставить одну на страницу
2. Привести DOCTYPE к HTML5 стандарту
3. Использовать относительные или протоколно-нейтральные URL
4. Добавить заголовки кеширования

Подсказка: Эти ошибки часто касаются производительности и стандартизации.

- **Вложенные формы** — Разделить формы, оставить одну на страницу
- **Несоответствие DOCTYPE** — Привести DOCTYPE к HTML5 стандарту
- **Смешение HTTP/HTTPS в ресурсах** — Использовать относительные или протоколно-нейтральные URL
- **Отсутствие кеширования статических ресурсов** — Добавить заголовки кеширования

 Шаг 16:

Миссия 16

🔧 При комплексной проверке проекта LocalAI выполняет фазы. Упорядочите:

1. Сбор статистики ошибок всех файлов
2. Группировка повторяющихся ошибок
3. Генерация шаблонных правок для групп
4. Применение и контроль результатов

Подсказка: Сначала собрать данные, затем обработать их, потом применить.

Ответ:

- Сбор статистики ошибок всех файлов
- Группировка повторяющихся ошибок
- Генерация шаблонных правок для групп
- Применение и контроль результатов

 Шаг 17:

Миссия 17

🔧 Финальная викторина: LocalAI завершает цикл аудита всего проекта. Расположите этапы:

1. Запуск синтаксического валидатора
2. Анализ семантики и структуры
3. Проверка атрибутов и доступности
4. Объединение и фильтрация патчей
5. Применение правок и финальная проверка

Подсказка: Полный цикл от проверки до финального результата.

Ответ:

- Запуск синтаксического валидатора
- Анализ семантики и структуры

- Проверка атрибутов и доступности
- Объединение и фильтрация патчей
- Применение правок и финальная проверка

Модуль 10 - Модуль 10. Расширения и интеграции

Урок 1 - Совместная работа Ollama и LocalAI

Урок 2 - Добавление моделей Codellama, StarCoder, DeepSeekCoder

Урок 3 - Расширение под CSSJS-анализ

Урок 4 - Экспорт отчётов в JSON, XML, Markdown

Урок 5 - Интеграция с VSCode и браузером

Урок 6 - Викторина 1

Шаг 01:

Миссия 1

LocalAI предлагает добавить атрибут alt к `<img>` , но картинка декоративная. Что правильнее?
Подсказка: Помните о доступности и рекомендациях для декоративных изображений.

Ответ:

- **Добавить alt="" (пустой)**
- Оставить без alt
- Добавить alt с описанием

Шаг 02:

Миссия 2

Вы вставили в href ссылку без протокола, LocalAI ругается: “Отсутствует протокол”. Что выбрать?

Подсказка: Для надёжных относительных ссылок протокол важен.

Ответ:

- **Добавить полный протокол, например https://**
- Оставить как есть
- Добавить только //

Шаг 03:

Миссия 3

LocalAI советует закрыть тег `<li>` , но в списке у вас пустой элемент для визуального разделителя. Что делать?

Подсказка: Даже пустой

- может требовать структурной корректности.

Ответ:

- **Закрыть тег**
-
- Оставить незакрытым

- Заменить на

🔧 Шаг 04:

Миссия 4

🔗 Сопоставьте корректный атрибут с его назначением, который LocalAI может рекомендовать:

Атрибуты:

Ответ:

- `rel="noopener"`
- `loading="lazy"`
- `role="button"`
- `aria-hidden="true"`

Назначения:

1. Для кнопки, заданной через тег, чтобы читатели восприняли её как кнопку
2. Откладывать загрузку изображений для оптимизации
3. Предотвратить доступ окну-орепер к родительской странице
4. Скрыть элемент от средств чтения экрана

Подсказка: Подумайте об SEO, производительности, безопасности и доступности.

- **`rel="noopener"`** — Предотвратить доступ окну-орепер к родительской странице
- **`loading="lazy"`** — Откладывать загрузку изображений для оптимизации
- **`role="button"`** — Для кнопки, заданной через тег, чтобы читатели восприняли её как кнопку
- **`aria-hidden="true"`** — Скрыть элемент от средств чтения экрана

🔧 Шаг 05:

Миссия 5

🔧 Отладка ошибок, указанных LocalAI: расположите шаги по порядку:

1. Посмотреть подсказку и сообщение LocalAI
2. Открыть HTML-файл в редакторе
3. Внедрить предложенные исправления
4. Проверить страницу в браузере и убедиться, что всё работает

Подсказка: Подумайте: сначала нужно увидеть, потом исправить, затем проверить.

Ответ:

- Посмотреть подсказку и сообщение LocalAI
- Открыть HTML-файл в редакторе
- Внедрить предложенные исправления
- Проверить страницу в браузере и убедиться, что всё работает

🔧 Шаг 06:

Миссия 6

🔗 При автоматической проверке LocalAI выявил пять предупреждений. Какой порядок действий наиболее логичен?

1. Сгруппировать предупреждения по типам (доступность, структура, стили)
2. Начать с самых критичных (ошибки, а не советы)

3. Исправлять по одному блоку, периодически проверяя результат
4. После всех изменений запустить повторную проверку LocalAI
5. Завершить проверкой в браузере и автоматическими тестами

Подсказка: Ориентируйтесь на приоритет и итеративный подход.

Ответ:

- Сгруппировать предупреждения по типам (доступность, структура, стили)
- Начать с самых критичных (ошибки, а не советы)
- Исправлять по одному блоку, периодически проверяя результат
- После всех изменений запустить повторную проверку LocalAI
- Завершить проверкой в браузере и автоматическими тестами

Шаг 07:

Миссия 7

🔧 При исправлениях LocalAI советует несколько изменений — какое логичное упорядочение?

1. Исправить семантику тегов
2. Добавить недостающие атрибуты (aria, lang и др.)
3. Удалить устаревшие атрибуты и стили inline
4. Запустить проверку снова и убедиться в отсутствии ошибок

Подсказка: Семантика влияет на доступность, атрибуты — на взаимодействие, стили — на внешний вид.

Ответ:

- Исправить семантику тегов
- Добавить недостающие атрибуты (aria, lang и др.)
- Удалить устаревшие атрибуты и стили inline
- Запустить проверку снова и убедиться в отсутствии ошибок

Шаг 08:

Миссия 8

📐 Расположите этапы интеграции LocalAI с редактором кода:

1. Установить плагин или расширение
2. Настроить правила проверок и уровни предупреждений
3. Запустить проверку текущего HTML-файла
4. Применить рекомендации и пересмотреть код
5. Повторно выполнить проверку и убедиться в чистоте HTML

Подсказка: Подумайте о конвейере разработки: установка → настройка → проверка → исправление → проверка.

Ответ:

- Установить плагин или расширение
- Настроить правила проверок и уровни предупреждений
- Запустить проверку текущего HTML-файла
- Применить рекомендации и пересмотреть код
- Повторно выполнить проверку и убедиться в чистоте HTML

Шаг 09:

Миссия 9

 У вас есть серия HTML-файлов проекта — как правильно прогнать LocalAI?

1. Выбрать основной файл шаблона (layout)
2. Прогнать проверку по шаблону и базовым страницам
3. Исправить ошибки в шаблоне, поскольку они затрагивают все файлы
4. Запустить проверку остальных файлов проекта
5. Исправить оставшиеся ошибки и повторно проверить весь проект

Подсказка: Ошибки в шаблоне часто влияют на множество страниц.

Ответ:

- Выбрать основной файл шаблона (layout)
- Прогнать проверку по шаблону и базовым страницам
- Исправить ошибки в шаблоне, поскольку они затрагивают все файлы
- Запустить проверку остальных файлов проекта
- Исправить оставшиеся ошибки и повторно проверить весь проект

Шаг 10:

Миссия 10

 При интеграции LocalAI в CI/CD: расположите этапы запуска и отклика:

1. Запуск анализа HTML в момент коммита
2. Генерация отчёта с ошибками и предупреждениями
3. Отказ сборки при критичных ошибках
4. Отправка отчёта разработчику или в PR
5. **При успешной проверке** — продолжение сборки и деплой

Подсказка: CI/CD — автоматизированный конвейер, ошибки должны останавливать процесс.

Ответ:

- Запуск анализа HTML в момент коммита
- Генерация отчёта с ошибками и предупреждениями
- Отказ сборки при критичных ошибках
- Отправка отчёта разработчику или в PR
- **При успешной проверке** — продолжение сборки и деплой

Шаг 11:

Миссия 11

 Полный рабочий цикл с LocalAI: упорядочите шаги от начала до конца:

1. Разработка HTML-страницы
2. Запуск LocalAI для анализа
3. Получение списка рекомендаций и предупреждений
4. Применение правок к коду
5. Повторный анализ LocalAI и браузерное тестирование
6. Финальный пуш в репозиторий

Подсказка: Это цикл: пишем — проверяем — исправляем — тестируем — публикуем.

Ответ:

- Разработка HTML-страницы
- Запуск LocalAI для анализа
- Получение списка рекомендаций и предупреждений
- Применение правок к коду
- Повторный анализ LocalAI и браузерное тестирование
- Финальный пуш в репозиторий

Урок 7 - Викторина 2

Шаг 01:

Миссия 1

LocalAI подсказал: «поставьте атрибут title у ссылок». Ваши ссылки — иконки без текста. Что лучше?

Подсказка: Атрибут title помогает при отсутствии текстового контента.

Ответ:

- **Добавить title с описанием**
- Оставить без title
- Добавить текстовую подпись рядом

Шаг 02:

Миссия 2

Вы подключили LocalAI как расширение браузера, и он предлагает «используйте crossorigin attribute» для . Это нужно?

Подсказка: Распределённые ресурсы и безопасность.

Ответ:

- **Добавить crossorigin, если скрипт загружается с другого домена**
- Всегда добавлять crossorigin
- Никогда не добавлять crossorigin

Шаг 03:

Миссия 3

LocalAI говорит: «добавьте мета-тег viewport ». На что это влияет?

Подсказка: Мобильная адаптивность важна для UX.

Ответ:

- **Обеспечивает масштабирование на мобильных устройствах**
- Добавляет кэширование
- Ускоряет загрузку на десктопе

Шаг 04:

Миссия 4

LocalAI советует убрать атрибут style из

. Вы используете цветовую стилизацию inline. Что правильнее?

Подсказка: CSS-отделение логики стилей от разметки.

Ответ:

- **Перенести стиль в CSS-файл**
- Оставить inline style
- Заменить на атрибут class и инлайн CSS

Шаг 05:

Миссия 5

 Сопоставьте тип интеграции LocalAI и его преимущество:

Типы:

Ответ:

- Плагин для VS Code
- Расширение браузера
- CI/CD интеграция
- Скрипт локального анализа

Преимущества:

1. Автоматическая проверка при пуше в репозиторий
2. Анализ HTML прямо в браузере при загрузке страницы
3. Подсветка ошибок в коде при разработке
4. Проверка вне сетевого соединения, локально

Подсказка: Подумайте, где удобно получить результаты анализа.

- **Плагин для VS Code** — Подсветка ошибок в коде при разработке
- **Расширение браузера** — Анализ HTML прямо в браузере при загрузке страницы
- **CI/CD интеграция** — Автоматическая проверка при пуше в репозиторий
- **Скрипт локального анализа** — Проверка вне сетевого соединения, локально

Шаг 06:

Миссия 6

 Сопоставьте режим проверки LocalAI и его описание:

Режимы:

Ответ:

- Lint-режим
- Автоисправление (autofix)
- Только предупреждения (warn-only)
- Строгий режим (strict)

Описание:

1. Исправляет некоторые ошибки автоматически
2. Выводит только предупреждения, не ошибки
3. Максимальная строгость, ошибки блокируют работу
4. Стандартная проверка без изменений

Подсказка: Представьте, как будет реагировать система в разных режимах.

- **Lint-режим** — Стандартная проверка без изменений
- **Автоисправление (autofix)** — Исправляет некоторые ошибки автоматически
- **Только предупреждения (warn-only)** — Выводит только предупреждения, не ошибки
- **Строгий режим (strict)** — Максимальная строгость, ошибки блокируют работу

Шаг 07:

Миссия 7

🗣️ LocalAI выдаёт три совета: «cache-control», «prefetch», «defer». Сопоставьте советы и их эффекты:

Советы:

Ответ:

- cache-control
- prefetch
- defer

Эффекты:

1. Запустить загрузку ресурса заранее
2. Управлять кэшированием ресурсов браузером
3. Отложить выполнение скриптов до загрузки DOM

Подсказка: Подумайте о производительности загрузки страницы.

- **cache-control** — Управлять кэшированием ресурсов браузером
- **prefetch** — Запустить загрузку ресурса заранее
- **defer** — Отложить выполнение скриптов до загрузки DOM

Шаг 08:

Миссия 8

🗣️ Сопоставьте команды CLI LocalAI и их назначение:

Команды:

Ответ:

- localai analyze
- localai fix
- localai config
- localai watch

Назначения:

1. Автоматически исправлять обнаруженные ошибки
2. Запуск проверки на изменяющиеся файлы в режиме наблюдения
3. Просмотр и редактирование конфигурации
4. Только проверить и вывести отчёт

Подсказка: CLI-утилиты часто имеют логичные названия.

- **localai analyze** — Только проверить и вывести отчёт
- **localai fix** — Автоматически исправлять обнаруженные ошибки
- **localai config** — Просмотр и редактирование конфигурации
- **localai watch** — Запуск проверки на изменяющиеся файлы в режиме наблюдения

Шаг 09:

Миссия 9

✂ Порядок включения LocalAI в проект от нуля:

1. Установить пакет или расширение
2. Сконфигурировать правила и уровни
3. Прогнать анализ на шаблонах и базовых страницах
4. Интегрировать в редактор и CI
5. Обучить команду пользоваться LocalAI

Подсказка: Подумайте: сначала установка, потом настройка, потом внедрение.

Ответ:

- Установить пакет или расширение
- Сконфигурировать правила и уровни
- Прогнать анализ на шаблонах и базовых страницах
- Интегрировать в редактор и CI
- Обучить команду пользоваться LocalAI

Шаг 10:

Миссия 10

📁 Как лучше встроить LocalAI в процесс ревью кода?

1. Добавить проверку LocalAI в Pull Request
2. Отправлять отчёт о проблемах автору PR
3. Обсудить спорные рекомендации вручную
4. После исправлений повторно запустить LocalAI
5. Слить PR, если ошибок не осталось

Подсказка: В ревью важно человеческое мнение + автоматическая поддержка.

Ответ:

- Добавить проверку LocalAI в Pull Request
- Отправлять отчёт о проблемах автору PR
- Обсудить спорные рекомендации вручную
- После исправлений повторно запустить LocalAI
- Слить PR, если ошибок не осталось

Шаг 11:

Миссия 11

🔍 Этапы адаптации LocalAI к новому проекту:

1. Запустить анализ «голого» HTML
2. Собрать рекомендации и категории
3. Настроить уровни предупреждений и отключить шумные правила

4. Прогнать повторный анализ с новой конфигурацией
5. Документировать правила для команды

Подсказка: Первые рекомендации могут быть избыточны — адаптация важна.

Ответ:

- Запустить анализ «голового» HTML
- Собрать рекомендации и категории
- Настроить уровни предупреждений и отключить шумные правила
- Прогнать повторный анализ с новой конфигурацией
- Документировать правила для команды

Шаг 12:

Миссия 12

⚙ При масштабировании проекта с LocalAI: упорядочите шаги:

1. Выделить базовую конфигурацию для всех модулей
2. Создать локальные overrides для специфики модулей
3. Запустить анализ для каждого модуля отдельно
4. Собрать единый отчёт по проекту
5. Проанализировать перекрёстные и глобальные проблемы

Подсказка: Большие проекты требуют деления на модули.

Ответ:

- Выделить базовую конфигурацию для всех модулей
- Создать локальные overrides для специфики модулей
- Запустить анализ для каждого модуля отдельно
- Собрать единый отчёт по проекту
- Проанализировать перекрёстные и глобальные проблемы

Шаг 13:

Миссия 13

🔁 Цикл использования LocalAI при доработке функционала:

1. Добавить новые блоки HTML
2. Прогнать LocalAI-анализ
3. Исправить ошибки и рекомендации
4. Тестировать взаимодействие с JS/CSS
5. Повторно прогнать LocalAI и завершить работу

Подсказка: После изменений важно не сломать связки.

Ответ:

- Добавить новые блоки HTML
- Прогнать LocalAI-анализ
- Исправить ошибки и рекомендации
- Тестировать взаимодействие с JS/CSS
- Повторно прогнать LocalAI и завершить работу

Шаг 14:

Миссия 14

📁 При добавлении новых HTML-компонентов: как встроить LocalAI проверки?

1. Добавить компонент в dev-ветку
2. Прогнать LocalAI локально по компоненту
3. Исправить рекомендации
4. Открыть PR с чистым HTML
5. LocalAI проверит при CI перед мержем

Подсказка: Проверки компонентов удобны при модульной архитектуре.

Ответ:

- Добавить компонент в dev-ветку
- Прогнать LocalAI локально по компоненту
- Исправить рекомендации
- Открыть PR с чистым HTML
- LocalAI проверит при CI перед мержем

💡 **Шаг 15:**

Миссия 15

🔗 При исследовании отчётов LocalAI: как поступить?

1. Составить список наиболее частых ошибок
2. Понять, какие рекомендации ложные (false positives)
3. Откорректировать конфигурацию, отключить шумные правила
4. Прогнать анализ заново с новой конфигурацией
5. Формировать стабильный набор правил для проекта

Подсказка: Анализ отчётов помогает «настроить» LocalAI под проект.

Ответ:

- Составить список наиболее частых ошибок
- Понять, какие рекомендации ложные (false positives)
- Откорректировать конфигурацию, отключить шумные правила
- Прогнать анализ заново с новой конфигурацией
- Формировать стабильный набор правил для проекта

💡 **Шаг 16:**

Миссия 16

🔄 Полный рабочий цикл LocalAI при разработке нового функционала:

1. Разработка разметки HTML
2. Локальный запуск LocalAI
3. Применение рекомендаций
4. Автотестирование UI и взаимодействий
5. Запуск LocalAI в CI
6. Деплой, если нет ошибок

Подсказка: Это полный путь от написания до публикации с проверками.

Ответ:

- Разработка разметки HTML
- Локальный запуск LocalAI

- Применение рекомендаций
- Автотестирование UI и взаимодействий
- Запуск LocalAI в CI
- Деплой, если нет ошибок